Annual Reviews in Control 56 (2023) 100902

Contents lists available at ScienceDirect

Annual Reviews in Control

journal homepage: www.elsevier.com/locate/arcontrol

Review article

A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems

Kuize Zhang

Department of Computer Science, University of Surrey, GU2 7XH, Guildford, UK

ARTICLE INFO

Keywords:
Discrete-event system
Labeled finite-state automaton
Inference
Concealment
Concurrent composition
Observer
Linear temporal logic

ABSTRACT

Discrete-event systems (DESs) usually consist of discrete states and transitions between them caused by spontaneous occurrences of labeled events. In this review article, we study DESs modeled by labeled (nondeterministic) finite-state automata (LFSAs). Due to the partially-observed feature of DESs, fundamental properties therein can be classified into two categories: state/event-inference-based properties (e.g., strong detectability, diagnosability, and predictability) and state-concealment-based properties (e.g., opacity). Intuitively, the former category describe whether one can use observed output sequences to infer the current and subsequent states, past occurrences of faulty events, or future certain occurrences of faulty events; while the latter describe whether one cannot use observed output sequences to infer whether secret states have been visited (that is, whether the DES can conceal the status that its secret states have been visited). Over the past two decades these properties were studied separately using different methods, and particularly, in most works inference-based properties were verified based on two fundamental assumptions of deadlock-freeness and divergence-freeness, where the former implies that an automaton will always run, the latter implies that an automaton has no reachable unobservable cycle, hence the running of such an automaton will always be eventually observed. In this article, for LFSAs, a unified concurrent-composition method is shown to verify all above inference-based and concealment-based properties, resulting in a unified mathematical framework for the two categories of properties. In addition, compared with the previous methods in the literature, the concurrent-composition method does not depend on assumptions and is currently the most efficient. Finally, based on concurrent composition, we represent the negations of the above inference-based properties as linear temporal logic (LTL) formulae; by combining the concurrent composition and an additional tool called observer (i.e., the classical powerset construction for LFSAs), we also represent the above concealment-based properties as LTL formulae. Although LTL formulae model checking algorithms do not provide more efficient verification for these inference-based and concealment-based properties, the obtained LTL formulae show common similarities among these properties.

1. Introduction

Discrete-event systems (DESs) usually consist of discrete states and transitions between them caused by spontaneous occurrences of labeled (aka partially-observed) events. Hence DESs are autonomous (they are not driven by external factors) and nonlinear (Wonham & Cai, 2019). Due to the partially-observed feature of DESs, observation-based fundamental properties can be classified into two categories: *inference*-based and *concealment*-based. The former means whether one can infer further information of a DES from the observations to the DES, so that these information can be used to do further study on the DES, e.g., synthesizing a controller to change several properties of the DES. While the latter refers to whether the DES can forbid some information from being leaked to external intruders, even though the intruders can see outputs/labels generated by the DES. Roughly speaking, the former is dual to the latter. The former category contain detectability, diagnosability, predictability, etc.; the latter contain opacity, etc. Detectability, diagnosability, and predictability mean whether one can use observed output sequences generated by a DES to determine its current states, past occurrences of faulty events, and future certain occurrences of faulty events; opacity refers to whether one cannot use observed output sequences to determine whether secret states have been visited.

In the thorough review on the history of DESs (Silva, 2018) which provided "an aerial view" of "five continents" for the development of

E-mail address: kuize.zhang@surrey.ac.uk.

https://doi.org/10.1016/j.arcontrol.2023.100902
Received 26 November 2022; Received in revised form 15 July 2023; Accepted 16 July 2023
Available online 2 August 2023
1367-5788/© 2023 Elsevier Ltd. All rights reserved.

the field presented roughly in chronological order, the above inference-based properties and concealment-based properties were formulated in the fourth continent which was called *a second turning decade* (since the end of the 1970s and the 1980s, "an extremely active period"). Lafortune et al. (2018) retrospect the origin and development of two properties — diagnosability and opacity formulated in the fourth continent: they introduced what led to formal definitions of diagnosability in the 1990s (Lin, 1994; Sampath et al., 1995) and opacity in the 2000s (Bryans et al., 2008, 2005; Mazaré, 2004; Saboori & Hadjicostis, 2007), and how these properties have "withstood the test of time". The two properties were both motivated from practical requirements. However, detectability seems to lack a practical motivation (Shu et al., 2007), which may be the reason why detectability did not receive so much attention as diagnosability. However, detectability can be believed to be the most fundamental property with respect to the partially-observed feature: a primary curiosity is to study whether one can determine internal states through observed output sequences. As a matter of fact, the state detection problem was studied the earliest among the problems related to formulation and verification of inference-based properties, and the study was motivated from thought experiments which was called Gedanken-experiment by Moore (1956) and the property studied therein is physically/philosophically the same as the notion of observability in linear control systems (Kalman, 1960). The systems considered by Moore (1956) are not DESs but sequential machines which were called Moore machines later. Moore machines are deterministic, finite-state, finite-input, finite-output dynamical systems, not necessarily linear or autonomous. The terminology — discrete-event system appeared later than Moore machines (Silva, 2018).

Over the past two decades, these inference-based and concealment-based properties were studied separately, but no intrinsic relations between them were revealed. In this paper, for DESs modeled by labeled (nondeterministic) finite-state automata (LFSAs), a unified *concurrent-composition* method is given to verify all of them, thus a unified mathematical framework is given to accommodate many inference-based and concealment-based properties. Roughly speaking, a concurrent composition collects all pairs of trajectories of two systems producing the same output sequence, in which observable transitions with the same output are synchronized but unobservable transitions interleave. For strong detectability, diagnosability, and predictability (which are inference-based properties), the concurrent compositions of two slight variants of an LFSA are constructed to do verification in polynomial time; for variants of opacity (which are concealment-based properties), the concurrent compositions of slight variants of the LFSA and the observers[1] of slight variants of the LFSA are constructed to do verification in exponential time, all in the size of the LFSA. Hence in a large extent, *from a complexity point of view, it is easy to infer something, but it is hard to conceal something.* The unified concurrent-composition method coincidently can be regarded as a resonance point of the second of the two key points of Silva (2018) on diversity of DESs: (1) to understand that scientific and technical differences enrich the domain and (2) efforts should be made to reduce unnecessary dispersion.

The concurrent-composition method shows advantages in verifying inference-based properties of DESs since it does not rely on any assumption. Existing results in employing the detector method (Shu & Lin, 2011) to verify strong detectability depends on two fundamental assumptions, i.e., deadlock-freeness which assumes an LFSA always running, and divergence-freeness which requires the running of an LFSA will always be eventually observed. These two assumptions are also needed in the twin-plant method (Jiang et al., 2001) and the verifier method (Genc & Lafortune, 2009; Yoo & Lafortune, 2002) for verifying diagnosability and predictability.

When being applied to verify concealment-based properties, the concurrent-composition method is more efficient than almost all methods in the literature, e.g., the initial-state-estimator method (Saboori & Hadjicostis, 2012), the two-way-observer method (Yin & Lafortune, 2017), the K-delay-trajectory-estimator-method (Falcone & Marchand, 2015), the K/Inf-step-recognizer method (Ma et al., 2021), and the method based on nonsecret specification automata and observers (Wintenberg et al., 2022). The only known exception lies in the fact that only two variants of opacity, current-state opacity and initial-state opacity, can be verified by directly using the notions of observer (Cassez et al., 2009; Saboori & Hadjicostis, 2007) and reverse observer (Wu & Lafortune, 2013) (see Section 6 for details).

2. Preliminaries

Notation Symbol $\mathbb{N}$ denotes the set of nonnegative integers. Let Σ denote an alphabet, i.e., a nonempty finite set for which every sequence of elements of Σ is a unique sequence of elements of Σ.[2] Elements of Σ are called *letters*. As usual, we use Σ^* to denote the set of *words* (i.e., finite-length sequences of letters) over Σ including the empty word ϵ, and use Σ^ω to denote the set of *configurations* (i.e., infinite-length sequences of letters) over Σ, respectively. $\Sigma^+ := \Sigma^* \setminus \{\epsilon\}$. A *(formal) language* is a subset of Σ^*. For a word $s \in \Sigma^*$, $\mathrm{last}(s)$ denotes its last letter, $|s|$ stands for its length. We set $|s'| = +\infty$ for all $s' \in \Sigma^\omega$. For $s \in \Sigma^+$ and $k \in \mathbb{N}$, s^k and s^ω denote the concatenations of k copies of s and infinitely many copies of s, respectively. Analogously, for two languages $L_1, L_2 \subset \Sigma^*$, their concatenation is defined by $L_1L_2 := \{e_1e_2 | e_1 \in L_1, e_2 \in L_2\}$. For a word (configuration) $s \in \Sigma^*(\Sigma^\omega)$, a word $s' \in \Sigma^*$ is called a *prefix* of s, denoted as $s' \sqsubset s$, if there exists another word (configuration) $s'' \in \Sigma^*(\Sigma^\omega)$ such that $s = s's''$. For two nonnegative integers $i \le j$, $[\![i, j]\!]$ denotes the set of all integers no less than i and no greater than j. For a set S, $|S|$ denotes its cardinality and 2^S its power set. Symbols $\subset$ and $\subsetneq$ denote the subset and strict subset relations, respectively. "$\forall\exists$" denotes a property in the form of "for all ... there exists ...". "$\exists\forall$" denotes a property in the form of "there exists ... for all ...". The property will be specified according to context.

2.1. Labeled finite-state automata

Definition 1. A *labeled finite-state automaton* (LFSA) is a sextuple

$$S = (Q, E, \delta, Q_0, \Sigma, \ell), \tag{1}$$

where

1. Q is a finite set of *states*,
2. E is an alphabet of *events*,
3. $\delta : Q \times E \to 2^Q$ is the *transition function* (equivalently represented by the *transition relation* $\delta \subset Q \times E \times Q$ such that $q' \in \delta(q, e)$ if and only if $(q, e, q') \in \delta$),
4. $Q_0 \subset Q$ is a set of *initial states*,
5. Σ is an alphabet of *outputs/labels*, and
6. $\ell : E \to \Sigma \cup \{\epsilon\}$ is the *labeling function*.

A transition $(q, e, q') \in \delta$ is interpreted as when S is in state q and event e occurs S transitions to state q'. The event set E can been rewritten as disjoint union of *observable* event set $E_o = \{e \in E | \ell(e) \in \Sigma\}$ and *unobservable* event set $E_{uo} = \{e \in E | \ell(e) = \epsilon\}$. When an observable event occurs, its label can be observed; when an unobservable event occurs, nothing can be observed. Transition function $\delta : Q \times E \to 2^Q$ is recursively extended to $\delta : Q \times E^* \to 2^Q$ as follows: for all $q \in Q$, $u \in E^*$, and $a \in \Sigma$, one has $\delta(q, \epsilon) = \{q\}$ and $\delta(q, ua) = \bigcup_{p \in \delta(q,u)} \delta(p, a)$; equivalently, transition relation $\delta \subset Q \times E \times Q$ is recursively extended to $\delta \subset Q \times E^* \times Q$ as follows: (1) for all $q, q' \in Q$, $(q, \epsilon, q') \in \delta$ if and only if $q = q'$; (2) for all $q, q' \in Q$, $s \in E^*$, and $e \in E$, one has $(q, se, q') \in \delta$,

[1] i.e., the standard powerset construction used for determinizing nondeterministic finite automata with ϵ-transitions (Rabin & Scott, 1959; Sipser, 1996).

[2] For example, $\{0, 00\}$ is not an alphabet since $000 = 0\ 00 = 00\ 0$.

also denoted by $q \xrightarrow{se} q'$, called *transition sequence* or *run*, if and only if, $(q, s, q''), (q'', e, q') \in \delta$ for some $q'' \in Q$.

Labeling function $\ell : E \to \Sigma \cup \{\epsilon\}$ is recursively extended to $\ell : E^* \cup E^\omega \to \Sigma^* \cup \Sigma^\omega$ as $\ell(e_1 e_2 \dots) = \ell(e_1)\ell(e_2)\dots$ and $\ell(\epsilon) = \epsilon$. For all $E' \subset E$, $\ell(E') := \{\ell(e) | e \in E'\}$. In the current paper, we only consider static labeling functions. In such labeling functions, the label at each letter is determined by the letter itself, e.g., for a word $e_1 \dots e_n \in E^*$, for each $i \in [\![1, n]\!]$, the label at e_i is determined by e_i itself. More complex labeling functions such as dynamic labeling functions and (m-)orwellian labeling functions were considered by Bryans et al. (2008), etc. In dynamic labeling functions, the label at each letter is determined by the letter itself and all its past letters, e.g., the label at e_i is determined by $e_1 \dots e_i$. In (m-)orwellian labeling functions, the label at a letter is determined by the letter itself and all its past and future (m) letters, e.g., the label at e_i is determined by $e_1 \dots e_n$ ($e_1 \dots e_i e_{i+1} \dots e_{i+m}$).

Transitions $x \xrightarrow{e} x'$ with $\ell(e) = \epsilon$ (resp., $\ell(e) \neq \epsilon$) are called *unobservable* (resp., *observable*). For $q \in Q$ and $s \in E^+$, (q, s, q) is called a *transition cycle* if $(q, s, q) \in \delta$. An *observable transition cycle* is defined by a transition cycle with at least one observable transition. Analogously an *unobservable transition cycle* is defined by a transition cycle with no observable transition. An LFSA is called *deterministic* if $|Q_0| = 1$ and for all $q, q', q'' \in Q$ and $e \in E$, if $(q, e, q'), (q, e, q'') \in \delta$ then $q' = q''$.

A state $q \in Q$ is called *live/deadlock-free* if $(q, e, q') \in \delta$ for some $e \in E$ and $q' \in Q$. S is called *live/deadlock-free* if each of its reachable states is live. *A state $q' \in Q$ is reachable from a state $q \in Q$* if there exists $s \in E^+$ such that $q \xrightarrow{s} q'$. *A subset Q' of Q is reachable from a state $q \in Q$* if some state of Q' is reachable from q. Similarly *a state $q \in Q$ is reachable from a subset Q' of Q* if q is reachable from some state of Q'. A state $q \in Q$ is called *reachable* (*in S*) if either $q \in Q_0$ or it is reachable from some initial state. For a transition $(q, e, q') \in \delta$, a transition $(q'', e', q''') \in \delta$ is called a *predecessor* of (q, e, q') if either $q = q'''$ or q is reachable from q'''; a transition (q'', e', q''') is called a *successor* of (q, e, q') if either $q'' = q'$ or q'' is reachable from q'. S is called *convergent/divergence-free* if there is no reachable, unobservable transition cycle.

The symbol $L(S) := \{s \in E^* | (\exists q_0 \in Q_0)(\exists q \in Q)[q_0 \xrightarrow{s} q]\}$ will be used to denote the set of *finite-length event sequences* (called *finite traces*) generated by S, $L^\omega(S) = \{e_1 e_2 \dots \in E^\omega | (\exists q_0 \in Q_0)(\exists q_1, q_2, \dots \in Q)[q_0 \xrightarrow{e_1} q_1 \xrightarrow{e_2} \cdots]\}$ will denote the set of *infinite-length event sequences* (also called *infinite traces*) generated by S. Finite traces and infinite traces are also called traces sometimes. For each $\sigma \in \Sigma^*$, $\mathcal{M}(S, \sigma)$ denotes the *current-state estimate*, i.e., the set of states that the system can be in when σ has just been generated, i.e., $\mathcal{M}(S, \sigma) := \{q \in Q | (\exists q_0 \in Q_0)(\exists s \in E^*)[(\ell(s) = \sigma) \wedge (q_0 \xrightarrow{s} q)]\}$. $\mathcal{L}(S)$ denotes the *language generated* by S, i.e., $\mathcal{L}(S) := \{\sigma \in \Sigma^* | \mathcal{M}(S, \sigma) \neq \emptyset\}$. $\mathcal{L}^\omega(S)$ denotes the *ω-language generated* by S, i.e., $\mathcal{L}^\omega(S) := \{\sigma \in \Sigma^\omega | (\exists s \in L^\omega(S))[\ell(s) = \sigma]\}$. For a subset $x \subset Q$ of states, its *unobservable reach* $\mathrm{UR}(x)$ is defined by $\bigcup_{q \in x} \bigcup_{s \in (E_{uo})^*} \delta(q, s)$. If an LFSA S is deadlock-free and divergence-free, then $L^\omega(S) \neq \emptyset$ and $\mathcal{L}^\omega(S) \neq \emptyset$. For a state $q \in Q$, its *observable reach under $\sigma \in \Sigma$* is defined by $\mathrm{Reach}_\sigma(q) := \{q' \in Q | (\exists e \in E_o)[((q, e, q') \in \delta) \wedge (\sigma = \ell(e))]\}$. For a subset $x \subset Q$, its observable reach under σ is $\mathrm{Reach}_\sigma(x) = \bigcup_{q \in x} \mathrm{Reach}_\sigma(q)$.

We give a first LFSA S_1 (shown in Fig. 1) to illustrate the above definitions. In the sequel, when showing an LFSA, we sometimes omit the label of an observable event if they are the same, e.g., see Fig. 6. Sometimes we also omit the label of an unobservable event but show that this event is unobservable in the corresponding figure caption, e.g., see Fig. 19. Therefore, in an LFSA used for illustration, for an event, if its label is not shown in the corresponding figure or the figure caption, then it is observable and its label is the same as itself. This setting applies to all LFSAs except for the basic tool — concurrent compositions (see Definition 2). When illustrating the concurrent composition of two automata, the key point lies in whether an event is observable or not, but not in what the label of an observable event is, so we will not show the labels of events, because it is easy to distinguish between observable events (having no ϵ-components) and unobservable events (having one ϵ-component), e.g., see Fig. 2(b).

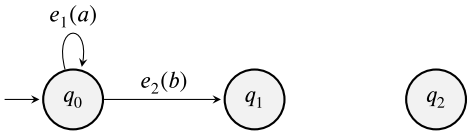


Fig. 1. LFSA S_1, where q_0 is the initial state (with an input arrow from nowhere), $\ell(e_1) = a$, $\ell(e_2) = b$.

Example 1. Consider the LFSA S_1 in Fig. 1. It is deterministic but not live (q_1 is dead). One sees $L(S_1) = \{(e_1)^n, (e_1)^n e_2 | n \geq 0\}$, $L^\omega(S_1) = \{(e_1)^\omega\}$, $\mathcal{L}(S_1) = \{a^n, a^n b | n \geq 0\}$, $\mathcal{L}^\omega(S_1) = \{a^\omega\}$, and $\mathcal{M}(S_1, b) = \{q_1\}$. States q_0 and q_1 are reachable, but state q_2 is not. q_1 is reachable from the transition cycle $q_0 \xrightarrow{e_1} q_0$. Transition $q_0 \xrightarrow{e_1} q_0$ is a predecessor of transition $q_0 \xrightarrow{e_2} q_1$, so $q_0 \xrightarrow{e_2} q_1$ is a successor of $q_0 \xrightarrow{e_1} q_0$.

2.2. The notion of concurrent composition

Next we introduce the main tool — concurrent composition. The concurrent-composition structure exactly arose from characterizing the *negation* of a strong version of detectability called *eventual strong detectability* (Zhang & Giua, 2020), where the eventual strong detectability is essentially different from and strictly weaker than the notion of *strong detectability* proposed by Shu et al. (2007). In the concurrent composition of two automata, observable transitions with the same label are synchronized, while unobservable transitions interleave. The way of using concurrent composition to verify strong versions of detectability brought in a *breakthrough* (Zhang, 2023a; Zhang & Giua, 2019; Zhang & Giua, 2020): The two assumptions of deadlock-freeness and divergence-freeness are not needed any more, where these two assumptions had been used since 2007 (Liu et al., 2019; Masopust, 2018; Shu & Lin, 2011, 2013a; Shu et al., 2007), because these authors verified strong versions of detectability themselves instead of verifying their negations as was done by Zhang (2023a), Zhang and Giua (2019), Zhang and Giua (2020). See Section 3 for details.

Definition 2 (*Zhang & Giua, 2020*)**.** Consider two LFSAs $S^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, we define the *concurrent composition* $\mathrm{CC}_{\mathrm{A}}(S^1, S^2)$ of S^1 and S^2 by

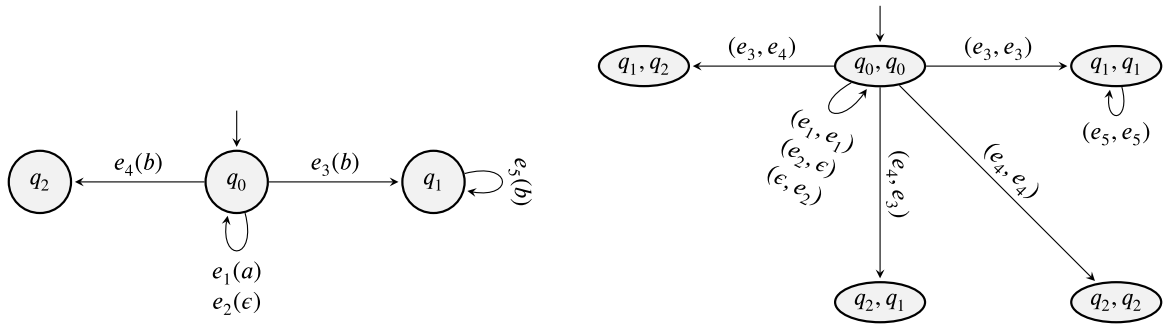
$$\mathrm{CC}_{\mathrm{A}}(S^1, S^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

where

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(\check{e}, \check{e}') | \check{e} \in E_{1o}, \check{e}' \in E_{2o}, \ell_1(\check{e}) = \ell_2(\check{e}')\}$, $E'_{uo} = \{(\check{e}, \epsilon) | \check{e} \in E_{1uo}\} \cup \{(\epsilon, \check{e}) | \check{e} \in E_{2uo}\}$;
3. for all $(\check{q}_1, \check{q}'_1), (\check{q}_2, \check{q}'_2) \in Q'$, $(\check{e}, \check{e}') \in E'_o$, $(\check{e}'', \epsilon) \in E'_{uo}$, and $(\epsilon, \check{e}''') \in E'_{uo}$,
   - $((\check{q}_1, \check{q}'_1), (\check{e}, \check{e}'), (\check{q}_2, \check{q}'_2)) \in \delta'$ if and only if $(\check{q}_1, \check{e}, \check{q}_2) \in \delta_1$, $(\check{q}'_1, \check{e}', \check{q}'_2) \in \delta_2$,
   - $((\check{q}_1, \check{q}'_1), (\check{e}'', \epsilon), (\check{q}_2, \check{q}'_2)) \in \delta'$ if and only if $(\check{q}_1, \check{e}'', \check{q}_2) \in \delta_1$, $\check{q}'_1 = \check{q}'_2$,
   - $((\check{q}_1, \check{q}'_1), (\epsilon, \check{e}'''), (\check{q}_2, \check{q}'_2)) \in \delta'$ if and only if $\check{q}_1 = \check{q}_2$, $(\check{q}'_1, \check{e}''', \check{q}'_2) \in \delta_2$;
4. $Q'_0 = Q_{01} \times Q_{02}$;
5. for all $(\check{e}, \check{e}') \in E'$, $\ell'((\check{e}, \check{e}')) := \ell_1(\check{e}) = \ell_2(\check{e}')$.

Particularly if $S^1 = S^2$, then $\mathrm{CC}_{\mathrm{A}}(S^1, S^2) =: \mathrm{CC}_{\mathrm{A}}(S^1)$ is called the *self-composition* of S^1.

For an event sequence $s' \in (E')^*$, we use $s'(L)$ and $s'(R)$ to denote its left and right components, respectively. Similar notation is applied to states of Q'. In addition, for every $s' \in (E')^*$, we use $\ell'(s')$ to



(a) $\mathcal{S}_2$. (b) $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_2)$.

Fig. 2. LFSA $\mathcal{S}_2$ (left) and its self-composition (right, only reachable states illustrated), where the events having an ϵ-component are unobservable, the other events are observable.
.

denote $\ell_1(s'(L))$ or $\ell_2(s'(R))$, since $\ell_1(s'(L)) = \ell_2(s'(R))$. In the above construction, $\mathrm{CC}_{\mathrm{A}}(S^1, S^2)$ aggregates all pairs of runs of S_1 and runs of S_2 that produce the same label sequence.

As mentioned above, when drawing a concurrent composition, we do not explicitly show labels of events, because it is crucial to distinguish between observable events (having no ϵ-components) and unobservable events (having exactly one ϵ-component), but it is not crucial what the label of an observable event is. See the following example. In the sequel, when illustrating a concurrent composition, we usually draw only necessary part of it (e.g., its reachable part).

Example 2. An LFSA $\mathcal{S}_2$ and its self-composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_2)$ are shown in Fig. 2.

When verifying concealment-based properties (Sections 6 and 7), except for the concurrent composition, we additionally need another basic tool — observer. The observer is the classical powerset construction used for determinizing nondeterministic finite automata with ϵ-transitions.

2.3. The notion of observer

Definition 3 (*Rabin & Scott, 1959; Sipser, 1996*). Consider an LFSA $\mathcal{S}$ (1). Its *observer* $\mathcal{S}_{\mathrm{obs}}$[3] is defined by a deterministic finite automaton

$$(Q_{\mathrm{obs}}, \ell(E_o), \delta_{\mathrm{obs}}, q_{0\,\mathrm{obs}}), \tag{3}$$

where

1. $Q_{\mathrm{obs}} = 2^Q$,
2. $\ell(E_o) = \ell(E) \setminus \{\epsilon\}$,
3. for all $x \in Q_{\mathrm{obs}}$ and $a \in \ell(E_o)$, $\delta_{\mathrm{obs}}(x, a) = \bigcup_{q \in x} \bigcup_{\substack{e_a \in E_o \\ \ell(e_a)=a \\ s \in (E_{uo})^*}} \delta(q, e_a s)$,
4. $q_{0\,\mathrm{obs}} = \bigcup_{q_0 \in Q_0} \bigcup_{s \in (E_{uo})^*} \delta(q_0, s)$ (i.e., $\mathrm{UR}(Q_0)$).

By definition, for all $a \in \ell(E_o)$, one has $\delta_{\mathrm{obs}}(\emptyset, a) = \emptyset$. The size of S_{obs} is $O(2^{|Q|}|\ell(E_o)|)$, the time consumption of computing S_{obs} is $O(2^{|Q|}|Q|^2|\ell(E_o)||E|)$: for every subset $x \subset Q$ and every label $a \in \ell(E_o)$, the time consumption of computing $\delta(x, a)$ is bounded (from above) by $|Q|^2|E_o| + |Q|^2|E_{uo}| = |Q|^2|E|$.

Example 3. Part of the observer $\mathcal{S}_{2\,\mathrm{obs}}$ of the LFSA $\mathcal{S}_2$ (Fig. 2) is shown in Fig. 3.

[3] the term "observer" dates back to Özveren and Willsky (1990), Shu et al. (2007).

2.4. Linear temporal logic formulae

Linear temporal logic (LTL) formulae are a class of temporal formulae that describe linear-time properties (Pnueli, 1977). In the last part of this article (Section 8), we will use the concurrent composition to transform plenty of (the negations of) inference-based properties and concealment-based properties of LFSAs into LTL formulae.

Let AP be a finite set of atomic propositions. An LTL formula φ over AP is defined recursively as

$$\varphi = \mathrm{true}\,|a|\varphi_1 \wedge \varphi_2|\neg\varphi|\bigcirc\varphi|\varphi_1 \mathsf{U} \varphi_2, \tag{4}$$

where $a \in \mathsf{AP}$, $\bigcirc$ means *next* ($\bigcirc\varphi$ means φ holds in the next time step), U means *until* ($\varphi_1 \mathsf{U} \varphi_2$ means φ_1 holds until φ_2 holds). The derived operators include $\vee, \rightarrow, \ldots, \Diamond\varphi := \mathrm{true}\,\mathsf{U}\varphi$ (φ *eventually* holds) and $\Box\varphi := \neg\Diamond\neg\varphi$ (φ *always* holds), etc.

Every LTL formula defines a linear-temporal property consisting of a set of infinite traces over 2^{AP}:

$$\mathrm{Words}(\varphi) := \left\{ \sigma \in \left(2^{\mathsf{AP}}\right)^{\omega} \middle| \sigma \vDash \varphi \right\}, \tag{5}$$

where $\sigma \vDash \varphi$ means that the infinite trace $\sigma := A_0 A_1 \ldots$ with each A_i in 2^{AP} satisfies formula φ.

Denote $\sigma = A_0 A_1 \ldots$, where $A_i \in 2^{\mathsf{AP}}$, $i \in \mathbb{N}$.

- $\sigma \vDash a \iff a \in A_0$.
- $\sigma \vDash \varphi_1 \wedge \varphi_2 \iff \sigma \vDash \varphi_1 \wedge \sigma \vDash \varphi_2$.
- $\sigma \vDash \neg\varphi \iff \sigma \nvDash \varphi$.
- $\sigma \vDash \bigcirc\varphi \iff A_1 A_2 \cdots \vDash \varphi$.
- $\sigma \vDash \varphi_1 \mathsf{U} \varphi_2 \iff$ there is $i \in \mathbb{N}$ such that $A_i A_{i+1} \cdots \vDash \varphi_2$ and for all $0 \le j < i$, $A_j A_{j+1} \cdots \vDash \varphi_1$.
- $\sigma \vDash \Diamond\varphi \iff$ there is $i \in \mathbb{N}$ such that $A_i A_{i+1} \cdots \vDash \varphi$.
- $\sigma \vDash \Box\varphi \iff$ for all $i \in \mathbb{N}$, $A_i A_{i+1} \cdots \vDash \varphi$.

The LTL model checking problem is as follows: Given a deadlock-free labeled finite transition system $TS = (S, Act, \rightarrow, I, \mathsf{AP}, L)$ and an LTL formula φ over AP, where S is a finite set of states with $I \subset S$ the set of initial states, Act is a finite alphabet of events, $\rightarrow \subset S \times Act \times S$ a finite set of transitions, and $L : Act \rightarrow 2^{\mathsf{AP}}$ the labeling function, check whether for each infinite run starting from I, its label sequence (an infinite sequence of subsets of AP) satisfies φ. Note that here the roles of states and events are swapped from the standard formulation of the LTL model checking problem shown in Baier and Katoen (2008), but complexity results remain the same. The LTL model checking problem is PSPACE-complete (Sistla & Clarke, 1985). Various subclasses of LTL formulae model checking problems were discussed in Bauland et al. (2011), Sistla and Clarke (1985), etc., e.g., for the subclass of LTL formulae that only contain temporal operator $\Diamond$ and propositional operator $\wedge$, the LTL model checking problem is already NP-hard; while for the subclass of LTL formulae that only contain temporal operator $\Diamond$ and propositional operator $\vee$, the LTL model checking problem belongs to NL.

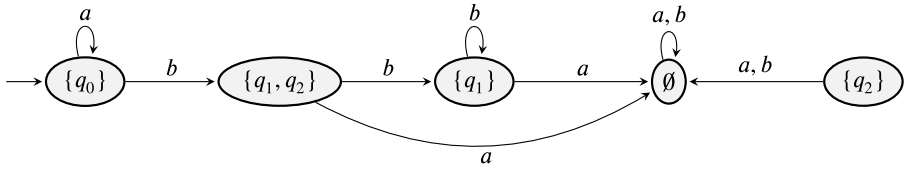


Fig. 3. Part of observer $S_{2\,\mathrm{obs}}$ of the LFSA S_2 in Fig. 2.

3. Detectability

In this section, we show how to use the concurrent-composition method to verify strong versions of detectability.

The study of the state detection problem dates back to the 1950s (Moore, 1956) in computer science and the 1960s (Kalman, 1963) in control science, respectively. In the former, Moore studied initial-state detection (called Gedanken-experiment) of finite-state machines which were called Moore machines later; in the latter, Kalman studied initial-state detection (called observability) of linear differential equations. The two seminal papers have induced many research branches in computer science and control, e.g., model-based testing of all kinds of reactive systems (Broy et al., 2005) in computer science and observability studies of all kinds of control systems, e.g., arranging from linear systems (Kalman, 1963; Wonham, 1985), to nonlinear systems (Conte et al., 2007; Isidori. Nonlinear Control Systems, 1995; Sontag, 1979), to switched systems (Tanwani et al., 2013), and also to networked systems (Angulo et al., 2019; Kibangou et al., 2016). Gill (1962) provided comprehensive discussions of all kinds of initial-state detection and current-state detection properties in Moore machines and Mealy machines,[4] by extending the original partition-based method proposed by Moore (1956). Gill (1962) seems be the first book that received attention from both control and computer science.

The state detection problem in DESs dates back to the 1980s (Özveren & Willsky, 1990; Ramadge, 1986). Two widely accepted fundamental notions are *strong detectability* and *weak detectability* proposed by Shu et al. (2007), where the former implies that there is a delay k such that for *each* trace generated by an LFSA, each prefix of its output sequence of length greater than k allows reconstructing the current state. The latter relaxes the former by changing the verbatim from *each* to *some*. When long-term behavior is considered, we let the above conditions apply to all generated infinite traces (in this case we call the notions ω-detectability); when short-term behavior is considered, we let them apply to all generated finite traces (in this case we call the notions $*$-detectability).

Definition 4 (ω-*Strong Detectability*). *(Sasi & Lin, 2018)* An LFSA S (1) is called *ω-strongly detectable* if there exists a positive integer k such that for each infinite trace $s \in L^\omega(S)$, $|\mathcal{M}(S,\sigma)| = 1$ for every prefix σ of $\ell(s)$ satisfying $|\sigma| > k$.

Definition 5 (ω-*Weak Detectability*). An LFSA S (1) is called *ω-weakly detectable* if $L^\omega(S) \neq \emptyset$ implies there exists an infinite trace $s \in L^\omega(S)$ such that for some positive integer k, $|\mathcal{M}(S,\sigma)| = 1$ for every prefix σ of $\ell(s)$ satisfying $|\sigma| > k$.

Intuitively, if an LFSA S is ω-strongly (resp., ω-weakly) detectable, then after some delay $k \in \mathbb{N}$, along each (resp., some if one exists) infinite generated trace, one can determine the corresponding state after observing more than k labels generated by the trace. In Definition 5, the phrase "$L^\omega(S) \neq \emptyset$ implies" is used to make Definition 5 weaker than Definition 4 so that the terms of "strong detectability" and "weak detectability" are logically related: At first glance, "strong detectability" should be stronger than "weak detectability". Without "$L^\omega(S) \neq \emptyset$ implies", an S with $L^\omega(S) = \emptyset$ would not be ω-weakly detectable, while such an S is vacuously ω-strongly detectable.

Note that ω-strong detectability and ω-weak detectability were initially defined on deterministic LFSAs that are deadlock-free and divergence-free by Shu et al. (2007). They were later defined on LFSAs that are deadlock-free and divergence-free by Shu and Lin (2011) and on general LFSAs by Sasi and Lin (2018), Zhang and Giua (2019), Zhang et al. (2020). The notions of strong detectability defined by Shu et al. (2007) (resp., Shu & Lin, 2011) are obtained from Definition 4 by removing the symbol ω and restricting S to be deadlock-free, divergence-free, and deterministic (resp., deadlock-free and divergence-free). Recall that a deadlock-free and divergence-free LFSA S satisfies $L^\omega(S) \neq \emptyset$ and $\mathcal{L}^\omega(S) \neq \emptyset$. The notions of weak detectability defined by Shu et al. (2007) (resp., Shu & Lin, 2011) are obtained from Definition 5 by removing the symbol ω and "$L^\omega(S) \neq \emptyset$ implies" and restricting S to be deadlock-free, divergence-free, and deterministic (resp., deadlock-free and divergence-free). The verification algorithms obtained by Shu and Lin (2011) are as follows:

A deadlock-free and divergence-free S is strongly detectable if and only if in its observer S_{obs}, every state reachable from a reachable cycle is a singleton; is weakly detectable if and only if in S_{obs}, there is a reachable cycle all of whose states are singletons. (A)

The two algorithms run in exponential time, since it costs exponential time to compute S_{obs}. After splitting the states of S_{obs} into subsets of cardinality 2,[5] a notion of detector S_{det} is obtained, and then a polynomial-time algorithm for verifying strong detectability is obtained (Shu & Lin, 2011):

A deadlock-free and divergence-free S is strongly detectable if and only if in its detector S_{det}, every state reachable from a reachable cycle is a singleton. (B)

This algorithm runs in polynomial time. See Table 1 as a summary.

Sasi and Lin (2018) studied detectability of an LFSA S within an imperfect single first-in, first-out communication channel[6] that allows event losses and bounded time delays, and obtained verification algorithms for strong detectability and weak detectability in the same form of (A). Unfortunately, the algorithms work correctly only under the two assumptions of deadlock-freeness and divergence-freeness, but do not work correctly without the two assumptions in general. The reason that the observer/detector methods do not work correctly without the two assumptions lies in that, there may exist two dead states reachable from some observable cycle under the same label sequence. Consider a variant $\bar{S}_2$ (Fig. 4(a)) obtained from LFSA S_2 (Fig. 2(a)) by removing the self-loop on q_1. Then part of $\bar{S}_{2\,\mathrm{obs}}$ (Fig. 4(b)) can be obtained from $S_{2\,\mathrm{obs}}$ (Fig. 3) by changing the transitions $\{q_1,q_2\} \xrightarrow{b} \{q_1\}$ and $\{q_1\} \xrightarrow{b} \{q_1\}$ to $\{q_1,q_2\} \xrightarrow{b} \emptyset$ and $\{q_1\} \xrightarrow{b} \emptyset$ (see Fig. 4), respectively. Since in $\bar{S}_{2\,\mathrm{obs}}$, state $\{q_1,q_2\}$ is reachable from the self-loop on $\{q_0\}$, the algorithm obtained in Sasi and Lin (2018) shows that $\bar{S}_2$ is not ω-strongly detectable. However, by definition, $\bar{S}_2$ satisfies Definition 4, that is, $\bar{S}_2$ is ω-strongly detectable.

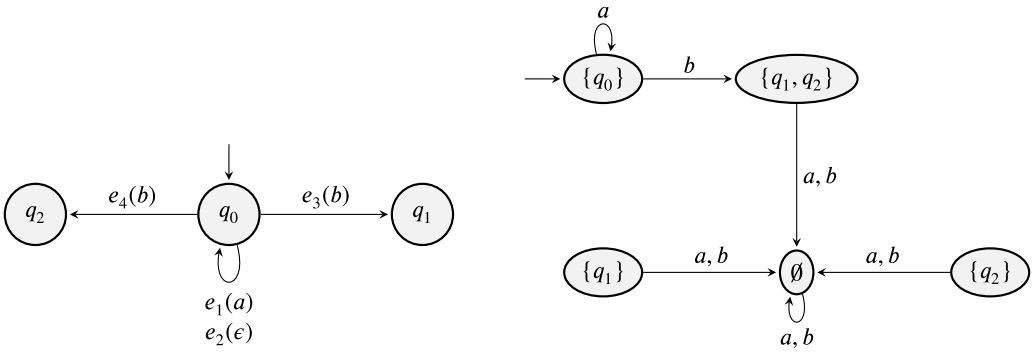
[4] Although Mealy machines are also deterministic finite-input, finite-state dynamical systems, their output functions depend on their states and inputs, while the output functions of Moore machines only depend on their states, so Mealy machines are more general than Moore machines.

[5] This idea was earlier used in Caines et al. (1988, 1991).

[6] In this article, for east of statement, we do not consider imperfect channels.

Table 1
Time complexity upper bounds on verifying various notions of detectability of LFSAs using different methods, where "¬" is short for "the negation of"; $\mathcal{S}$ denotes an LFSA, "d.d." is short for "deadlock-free (i.e., live), divergence-free", $|Q|$, $|E|$, and $|\ell(E_o)|$ denote the numbers of states, events, and distinct labels of the observable states of an LFSA, respectively. "c." is short for "complete". In order not to depend on the two assumptions of deadlock-freeness and divergence-freeness, the square on $|E|$ is indispensable. The independence of the two assumptions was derived from verifying the negations of strong versions of detectability. For deterministic $\mathcal{S}$, the above complexity upper bound for the detector method degenerates to $O(\frac{1}{2}|\ell(E_o)||E||Q|^3)$, the complexity upper bound for the concurrent-composition method degenerates to $O(|Q|^2|E|^2)$.

	ω-strong detectability		ω-weak detectability
d.d. $\mathcal{S}$	$O(\frac{1}{2}\|\ell(E_o)\|\|E\|\|Q\|^4)$ (Shu & Lin, 2011) detector NL-c. (Masopust, 2018)		$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Shu et al., 2007) observer PSPACE-c. (Zhang, 2017)
	¬ ω-strong detectability	¬ ω-eventual strong detectability	ω-weak detectability
$\mathcal{S}$	$O(\|Q\|^4\|E\|^2)$ (Zhang & Giua, 2019) concurrent composition NL (Zhang, 2021)	$O(\|Q\|^4\|E\|^2)$ (Zhang & Zhang, 2022) concurrent composition	PSPACE (Zhang & Giua, 2020) observer



(a) $\bar{\mathcal{S}}_2$. (b) Part of $\bar{\mathcal{S}}_{2\,\mathrm{obs}}$.

Fig. 4. LFSA $\bar{\mathcal{S}}_2$ (left) and part of its observer $\bar{\mathcal{S}}_{2\,\mathrm{obs}}$ (right). .

Observe that only a small number of LFSAs are deadlock-free and divergence-free, the observer/detector-based detectability verification methods derived by Sasi and Lin (2018), Shu and Lin (2011), Shu et al. (2007) have limitations.

The first detectability verification algorithm that does not depend on assumptions was designed by Zhang and Giua (2019), Zhang and Giua (2020) via verifying the negation of strong detectability. More importantly, from the procedure of verifying the negation, the basic tool — concurrent composition was proposed. Based on the concurrent composition, fundamental development in DESs has been obtained, which will be partially shown in the article. The concurrent-composition method is currently the unique method that can verify Definition 4 correctly. Furthermore, in Zhang and Feng (2020), also based on the concurrent-composition method, a polynomial-time algorithm was given to verify Definition 4 of LFSAs within an imperfect single first-in, first-out communication channel that allows event losses and bounded time delays. For consistency with writing, we do not introduce the result obtained in Zhang and Feng (2020) in this article.

As mentioned above, in order to describe short-term behavior, we let the definitions of detectability apply to all generated finite traces, and formulate the definitions as follows.

Definition 6 (*$*$-Strong Detectability*). An LFSA $\mathcal{S}$ (1) is called *$*$-strongly detectable* if there exists a positive integer k such that for each finite trace $s \in L(\mathcal{S})$, $|\mathcal{M}(\mathcal{S}, \sigma)| = 1$ for every prefix σ of $\ell(s)$ satisfying $|\sigma| > k$.

Definition 7 (*$*$-Weak Detectability*). An LFSA $\mathcal{S}$ (1) is called *$*$-weakly detectable* if $L(\mathcal{S}) \neq \emptyset$ implies there exists a finite trace $s \in L(\mathcal{S})$ such that for some positive integer k, $|\mathcal{M}(\mathcal{S}, \sigma)| = 1$ for every prefix σ of $\ell(s)$ satisfying $|\sigma| > k$.

Intuitively, if an LFSA $\mathcal{S}$ is $*$-strongly (resp., $*$-weakly) detectable, then after some delay $k \in \mathbb{N}$, along each (resp., some if one exists) finite generated trace, one can determine the corresponding state after observing more than k labels generated by the trace. Similar to "$L^\omega(\mathcal{S}) \neq \emptyset$ implies" in Definition 5, the phrase "$L(\mathcal{S}) \neq \emptyset$ implies" in Definition 7 guarantees that Definition 7 is weaker than Definition 6.

As mentioned above, a crucial step to verifying strong detectability without any assumption is to characterize its negation.

Proposition 3.1 (*Zhang et al., 2020*). *An LFSA $\mathcal{S}$ (1) is not ω-strongly detectable (resp., $*$-strongly detectable) if and only if for every positive integer k there exists an infinite (resp., finite) trace $s \in L^\omega(\mathcal{S})$ (resp., $s \in L(\mathcal{S})$) such that $|\mathcal{M}(\mathcal{S}, \sigma)| > 1$ for some prefix σ of $\ell(s)$ satisfying $|\sigma| > k$.*

With the notion of self-composition of an LFSA $\mathcal{S}$, we give necessary and sufficient conditions for the negations of two versions of strong detectability, without any assumption.

Theorem 3.2 (*Zhang & Giua, 2019*). *An LFSA $\mathcal{S}$ (1) is not $*$-strongly detectable if and only if in $\mathrm{CC_A}(\mathcal{S})$, there exists a run*

$$q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{s'_2} q'_1 \xrightarrow{s'_3} q'_2 \tag{6}$$

satisfying

$$q'_0 \in Q'_0;\ q'_1, q'_2 \in Q';\ s'_1, s'_2, s'_3 \in (E')^*; \tag{7a}$$

$$\ell(s'_2) \in \Sigma^+;\ q'_2(L) \neq q'_2(R). \tag{7b}$$

An LFSA $\mathcal{S}$ is not ω-strongly detectable if and only if in $\mathrm{CC}_{\mathrm{A}}(\mathcal{S})$, there exists a run (6) satisfying (7) and in $\mathcal{S}$, there exists a transition cycle reachable from $q'_2(L)$.

Proof. We use Proposition 3.1 to prove this theorem. We first consider $*$-strong detectability.

"only if": Assume $\mathcal{S}$ is not $*$-strongly detectable. Then by Proposition 3.1, choose $k = |Q|^2$, there exists $s_k \in L(\mathcal{S})$ and $\sigma \in \Sigma^+$ such that $\mathcal{M}(\mathcal{S}, \sigma) > 1$, $\sigma \sqsubset \ell(s)$, and $|\sigma| > k$. Then in $\mathrm{CC}_{\mathrm{A}}(\mathcal{S})$, there exists a run $q'_0 \xrightarrow{s'} q'$ such that $q'_0 \in Q'_0$, $\ell(s') = \sigma$ and $q'(L) \neq q'(R)$. Since $|\sigma| > |Q|^2$ and there exist at most $|Q|^2$ distinct states in $\mathrm{CC}_{\mathrm{A}}(\mathcal{S})$, by the pigeonhole principle,[7] the run $q'_0 \xrightarrow{s'} q'$ can be rewritten as $q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{s'_2} q'_1 \xrightarrow{s'_3} q'$, where $\ell(s'_2) \in \Sigma^+$.

"if": Assume in $\mathrm{CC}_{\mathrm{A}}(\mathcal{S})$ there exists a run (6) satisfying (7). We choose trace $s := s'_1(L)(s'_2(L))^{k+1}s'_3(L) \in L(\mathcal{S})$, then $|\ell(s)| > k$ and $|\mathcal{M}(\mathcal{S}, \ell(s))| > 1$. By Proposition 3.1, $\mathcal{S}$ is not $*$-strongly detectable.

We second consider ω-strong detectability. Because a transition cycle reachable from $q'_2(L)$ can be repeated arbitrarily often, resulting in an infinite-length run starting from $q'_2(L)$. Then based on the above argument for $*$-strong detectability, the necessary and sufficient condition for ω-strong detectability also holds. □

Example 4. Reconsider the LFSA $\mathcal{S}_2$ in Fig. 2(a) and its self-composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_2)$ in Fig. 2(b). In $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_2)$, one sees a run

$$(q_0, q_0) \xrightarrow{(e_1,e_1)} (q_0, q_0) \xrightarrow{(e_3,e_4)} (q_1, q_2)$$

such that $\ell((e_1, e_1)) = a$ is of positive length and $q_1 \neq q_2$. Then by Theorem 3.2, $\mathcal{S}_2$ is not $*$-strongly detectable. In addition, in $\mathcal{S}_2$, there is a self-loop on q_1, hence also by Theorem 3.2, $\mathcal{S}_2$ is not ω-strongly detectable.

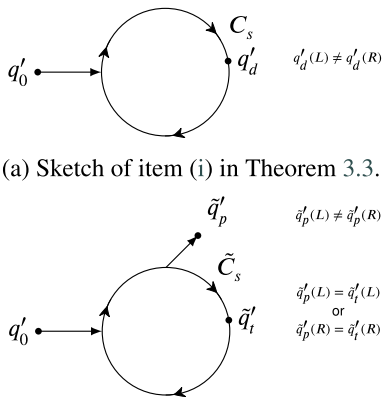
Before ending this section, we introduce another strong version of detectability — *eventual strong detectability* proposed by Zhang and Giua (2020) — which is strictly weaker than strong detectability. Eventual strong detectability implies that for each trace s generated by an LFSA, there is a delay k_s such that each prefix of the label sequence of s of length greater than k_s allows reconstructing the current state. Eventual strong detectability is weaker than strong detectability because the delay k_s depends on s. Eventual strong detectability is in the form of $\forall\exists$ while strong detectability is in the form of $\exists\forall$ (see the end of the Notation paragraph on Section 2).

Definition 8 (*ω-Eventual Strong Detectability*)**.** *(Zhang & Giua, 2020)* An LFSA $\mathcal{S}$ is called *ω-eventually strongly detectable* if for each infinite trace $s \in L^{\omega}(\mathcal{S})$, there exists a positive integer k_s (depending on s) such that $|\mathcal{M}(\mathcal{S}, \sigma)| = 1$ for every prefix σ of $\ell(s)$ satisfying $|\sigma| > k_s$.

Definition 9 (*$*$-Eventual Strong Detectability*)**.** An LFSA $\mathcal{S}$ is called *$*$-eventually strongly detectable* if for each finite trace $s \in L(\mathcal{S})$, there exists a positive integer k_s (depending on s) such that $|\mathcal{M}(\mathcal{S}, \sigma)| = 1$ for every prefix σ of $\ell(s)$ satisfying $|\sigma| > k_s$.

From now on we only study ω-eventual strong detectability. $*$-eventual strong detectability is not interesting, because every LFSA is $*$-eventually strongly detectable. Note that in Zhang and Giua (2020), Zhang and Zhang (2022), the symbol "ω-" was not used.

In Zhang and Giua (2020), a polynomial-time verification algorithm for the negation of ω-eventual strong detectability of LFSAs was designed by developing three tools — concurrent composition, observation automaton, and bifurcation automaton. Later in Zhang and Zhang (2022), ω-eventual strong detectability was extended to *labeled weighted automata over monoids*, and a necessary and sufficient condition for the negation of ω-eventual strong detectability of labeled weighted automata over monoids was given only based on an extended version of concurrent composition adapted to labeled weighted automata over monoids, where the extended version of concurrent composition was proposed by Zhang (2022). Particularly, for labeled weighted automata over the monoid $(\mathbb{Q}^k, +)$, the condition can be verified in NP; and for LFSAs, the condition can be verified in polynomial time (Zhang & Zhang, 2022). The study on labeled weighted automata over monoids has just started. Two basic tools – concurrent composition and observer have been developed and computed for labeled weighted automata over the monoid $(\mathbb{Q}^k, +)$ by developing original techniques (Zhang, 2022).



(a) Sketch of item (i) in Theorem 3.3.

(b) Sketch of item (ii) in Theorem 3.3.

Fig. 5. Sketch of the proof of Theorem 3.3.

We state the result on the necessary and sufficient condition for ω-eventual strong detectability of LFSAs obtained in Zhang and Zhang (2022). This condition is simpler (though not easier to get) than the one obtained in Zhang and Giua (2020).

Theorem 3.3 (*Zhang & Zhang, 2022*)**.** *An LFSA $\mathcal{S}$ is not ω-eventually strongly detectable if and only if one of the following two items holds.*

(i) In $\mathrm{CC}_{\mathrm{A}}(\mathcal{S})$ (see Fig. 5(a) for illustration), there exists a reachable, observable cycle C_s that contains a state $q'_d = (q'_d(L), q'_d(R))$ satisfying $q'_d(L) \neq q'_d(R)$.

(ii) In $\mathrm{CC}_{\mathrm{A}}(\mathcal{S})$ (see Fig. 5(b) for illustration),

- *(a) there exists a reachable, observable cycle $\tilde{C}_s$ such that every state $q' \in \tilde{C}_s$ satisfies $q'(L) = q'(R)$,*
- *(b) there exists a state $\tilde{q}'_p$ with $\tilde{q}'_p(L) \neq \tilde{q}'_p(R)$ reachable from cycle $\tilde{C}_s$,*
- *(c) there exists a state $\tilde{q}'_t \in \tilde{C}_s$ such that either $\tilde{q}'_p(L) = \tilde{q}'_t(L)$ or $\tilde{q}'_p(R) = \tilde{q}'_t(R)$.*

By comparing Theorems 3.2 and 3.3, one can see the condition for ω-eventual strong detectability is much more complex than that for ω-strong detectability. This also shows that it is much more difficult to obtain a necessary and sufficient condition for ω-eventual strong detectability. We omit the intricate proof of Theorem 3.3. From this comparison, one may expect that ω-eventual strong detectability is strictly weaker than ω-strong detectability. Indeed so, but an example that is ω-eventually strongly detectable but not ω-strongly detectable is not easy to find. When writing the paper (Zhang & Giua, 2020), first, a labeled Petri net with inhibitor arcs that is ω-eventually strongly detectable but not ω-strongly detectable was found, but it was not known if these two definitions are equivalent for labeled Petri nets then. After a long time, a labeled Petri net (shown in Zhang & Giua, 2020, Figure 4) that is ω-eventually strongly detectable but not ω-strongly

[7] If n items are put into m containers with $n > m$, then at least one container must contain more than one item.

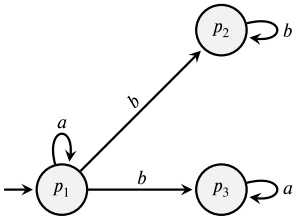


Fig. 6. An LFSA that is ω-eventually strongly detectable, but not ω-strongly detectable.

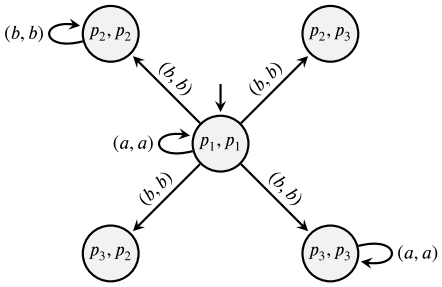


Fig. 7. The self-composition of the LFSA in Fig. 6.

detectable was finally found, and coincidently, the net is an LFSA. Then we knew that the two definitions are not equivalent even for LFSAs. This is an essential difference between detectability and diagnosability, because for diagnosability, the two forms $\forall\exists$ and $\exists\forall$ are equivalent. Details for diagnosability will be shown in Section 4.

Example 5. An LFSA that is ω-eventually strongly detectable but not ω-strongly detectable is shown in Fig. 6, where the LFSA is equivalent to the labeled Petri net in Zhang and Giua (2020, Figure 4). The self-composition of the LFSA in Fig. 6 is shown in Fig. 7. The self-composition satisfies neither (i) nor (ii) in Theorem 3.3.

Remark 3.4. In this section, we only discuss strong detectability and eventual strong detectability. The negations of many different types of strong versions of detectability in the literature can be equivalently represented by concurrent composition or their slight variants without any assumptions, e.g., delayed strong versions of detectability (Shu & Lin, 2013a), initial strong versions of detectability (Shu & Lin, 2013b), strong periodic detectability (Shu & Lin, 2011). While in these papers, all results are based on the two assumptions of deadlock-freeness and divergence-freeness, because the verification algorithms are slight modifications of the one proposed by Shu and Lin (2011) based on the notion of detector. The two-way verifier method used for verifying delayed strong detectability used in Liu et al. (2020) is also based on the two assumptions. A slight variant of the concurrent composition was constructed and proven to have stronger expressive power than the detector, and then the variant was used to verify the negation of strong periodic detectability of LFSAs without any assumption in polynomial time (Zhang, 2023b).

4. Diagnosability

In order to define *diagnosability* for an LFSA $\mathcal{S}$ (1), we specify a subset $E_f \subset E$ of faulty events. Diagnosability describes whether one can use an observed output sequence to determine whether some faulty event has occurred. For an event sequence $s \in E^*$, with slight abuse of notation, $E_f \in s$ denotes that some element of E_f appears in s.

Definition 10 (*Diagnosability*). *(Sampath et al., 1995)* Consider an LFSA $\mathcal{S}$ (1) and a subset $E_f \subset E$ of faulty events. $\mathcal{S}$ is called E_f*-diagnosable* if

$$(\exists k \in \mathbb{N})(\forall s \in L(\mathcal{S}) \cap E^* E_f)(\forall s' : ss' \in L(\mathcal{S})) \tag{8a}$$

$$[(|s'| > k) \implies \mathbf{D}], \tag{8b}$$

where

$$\mathbf{D} = (\forall s'' \in L(\mathcal{S}))[(\ell(s'') = \ell(ss')) \implies (E_f \in s'')]. \tag{9}$$

Intuitively, if $\mathcal{S}$ is E_f-diagnosable, then once a faulty event (e.g., the last event in s) occurs, one can be sure that some faulty event has occurred after at least k subsequent events (e.g., s') occur by observing output sequences.

In Sampath et al. (1995), diagnosability was defined on deterministic LFSAs that are deadlock-free and divergence-free. Sampath et al. (1995) proposed a diagnoser[8] method to verify diagnosability. The diagnoser of an LFSA records state estimates along observed output sequences and also records fault propagation along transitions of states of the LFSA. The same as the observer mentioned above for strong detectability, the size of the diagnoser of an LFSA is also exponential in the size of the LFSA, and it also takes exponential time to compute the diagnoser. Diagnosability is verifiable by a relatively simple cycle condition on the diagnoser. Hence diagnosability can be verified in exponential time. Also the same as the case that the observer method depends on the two assumptions of deadlock-freeness and divergence-freeness when being applied to verify detectability (Shu et al., 2007), the diagnoser method also depends on the two assumptions when being applied to verify diagnosability. Later, a twin-plant method with polynomial complexity was proposed by Jiang et al. (2001) to verify diagnosability in polynomial time. Because in a twin-plant, only observable transitions are synchronized, the method also depends on the two assumptions. One year later, Yoo and Lafortune (2002) proposed a verifier method to verify diagnosability in polynomial time, where in a verifier, observable transitions are synchronized, unobservable transitions are also considered but their events' positions (left or right) are neglected, so that the method also depends on the two assumptions. The essential reason why the three methods of diagnoser, twin-plant, and verifier depend on the two assumptions lies in that these methods were derived from verifying diagnosability *itself* instead of its negation. See Appendix for details. From then on, in many papers, verification of all kinds of variants of diagnosability depends on the two assumptions. The two assumptions were removed by Cassez and Tripakis (2008) by using a generalized version of the twin-plant structure to verify the *negation* of diagnosability in polynomial time, where in the generalized version of the twin-plant, observable transitions are synchronized, unobservable transitions are also considered but their events' positions (left or right) are also considered. Unfortunately, the automatic control community was not aware of the method proposed by Cassez and Tripakis (2008), most results published in control conferences and journals after the year 2008 on various definitions of diagnosability of LFSAs still depend on the two assumptions. The generalized version of the twin-plant structure and the concurrent-composition structure (Zhang & Giua, 2020) were proposed in a similar way: they were proposed by characterizing the negations of inference-based properties. The results in Cassez and Tripakis (2008), Zhang and Giua (2020) were worked out independently. After publishing papers (Zhang & Giua, 2019; Zhang & Giua, 2020), the writer started to read DES papers in computer science journals and conferences, and noticed the paper (Cassez & Tripakis, 2008). After that, the writer also noticed that a structure equivalent to the generalized twin-plant proposed by Cassez and Tripakis (2008) was also independently found in Bérard et al. (2018) in which the negation of diagnosability was verified. That is, the idea of verifying the

[8] a slight variant of the powerset construction used for determinizing nondeterministic finite automata with ϵ-transitions (Rabin & Scott, 1959; Sipser, 1996).

Table 2
Time complexity upper bounds on verifying diagnosability of LFSAs using different methods, where "¬" is short for "the negation of", S denotes an LFSA, "d.d." is short for "deadlock-free (i.e., live) and divergence-free", $|Q|$, $|E|$, and $|\ell(E_o)|$ denote the numbers of states, events, and distinct labels of the observable states of an LFSA, respectively. In order not to depend on the two assumptions of deadlock-freeness and divergence-freeness, the square on $|E|$ is indispensable. The independence of the two assumptions was derived from verifying the negation of diagnosability. $16|Q|^4$ (resp., $4|Q|^4$) means that in the corresponding verification structure, the states contain 4 (resp., 3) components, where 2 of them are states of S, the other 2 (resp., 1) are Y/N. The states of a concurrent composition are pairs of states of S. For deterministic S, the above complexity upper bounds degenerate by reducing 16, 4, and $|Q|^4$ to their positive square roots. The works in Bérard et al. (2018), Cassez and Tripakis (2008), Zhang (2021) were got independently.

	diagnosability	diagnosability	diagnosability
d.d. S	$O(16\|Q\|^4\|E\|)$ (Jiang et al., 2001) twin-plant	$O(16\|Q\|^4\|E\|)$ (Yoo & Lafortune, 2002) verifier	$O(4^{\|Q\|+1}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Sampath et al., 1995) diagnoser
	¬ diagnosability	¬ diagnosability	¬ diagnosability
S	$O(16\|Q\|^4\|E\|^2)$ (Cassez & Tripakis, 2008) generalized twin-plant	$O(4\|Q\|^4\|E\|^2)$ (Bérard et al., 2018) generalized twin-plant	$O(\|Q\|^4\|E\|^2)$ (Zhang, 2021) concurrent composition

negation of some inference-based property was independently proposed by Bérard et al. (2018), Cassez and Tripakis (2008), Zhang and Giua (2020), resulting in verification of inference-based properties without any assumption.

Remark 4.1. The technique of removing the two assumptions through model transformations (the most widely used technique is adding an unobservable self-loop on dead states) has been widely used in diagnosability studies in LFSAs, e.g., in Basilio et al. (2021), Qiu and Kumar (2006) and the references therein. However, this technique works only when the original definition of diagnosability proposed in Sampath et al. (1995) has been changed, where this change was clearly shown in Qiu and Kumar (2006, Definition 1) and Qiu and Kumar (2006, Remark 1). However, in Basilio et al. (2021), it was shown that the above technique can remove the two assumptions without indicating that the definition of diagnosability needs to be changed, which is technically incorrect. In Remark 3.20 of Zhang (2021), it was shown why this technique does not preserve the original definition of diagnosability in Sampath et al. (1995).

In order to verify E_f-diagnosability of S, we use the concurrent composition $\mathrm{CC_A}(S_\mathrm{f}, S_\mathrm{n})$ of the faulty subautomaton S_f and the normal subautomaton S_n, where S_f is obtained from S by only keeping reachable faulty transitions and all their predecessors and successors, S_n is obtained from S by removing all faulty transitions and subsequently removing all unreachable states and the transitions attached to them. $\mathrm{CC_A}(S_\mathrm{f}, S_\mathrm{n})$ is computed similarly as in Definition 2. It is similar to but simpler than the generalized versions of the twin-plant proposed by Bérard et al. (2018), Cassez and Tripakis (2008), because in the two versions of generalized twin-plant, the states contain additional components with entries F/N to mark whether some faulty event has occurred (e.g., see Appendix). However, in our concurrent composition used for verifying diagnosability shown as below, the states only contain two components. That is, F/N is not indispensable in order to verify diagnosability, the adoption of the F/N component in the two versions of generalized twin-plant in Bérard et al. (2018), Cassez and Tripakis (2008) is consistent with the style of the twin-plant (Jiang et al., 2001) and verifier (Yoo & Lafortune, 2002), in which the states also contain F/N components. Therefore, our method is a little more efficient. See Table 2 as a summary.

Theorem 4.2 (*Zhang, 2021*). *Consider an LFSA S (1) and a subset $E_f \subset E$ of faulty events. S is not E_f-diagnosable if and only if in $\mathrm{CC_A}(S_\mathrm{f}, S_\mathrm{n})$, there exists a run*

$$q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{e'} q'_2 \xrightarrow{s'_2} q'_3 \xrightarrow{s'_3} q'_3 \tag{10}$$

satisfying

$$q'_0 \in Q'_0;\ e'(L) \in E_f;\ |s'_3(L)| > 0. \tag{11}$$

Proof. By definition, S is not E_f-diagnosable if and only if

$$(\forall k \in \mathbb{N})(\exists s_k \in L(S) \cap E^* E_f)$$
$$(\exists s'_k : s_k s'_k \in L(S))(\exists s''_k \in L(S))$$
$$[(|s'_k| > k) \wedge (\ell(s''_k) = \ell(s_k s'_k)) \wedge (E_f \notin s''_k)].$$

Choose sufficiently large k, by the finiteness of the number of states of S and the pigeonhole principle, S is not E_f-diagnosable if and only if in $\mathrm{CC_A}(S_\mathrm{f}, S_\mathrm{n})$, there exists a run (10) satisfying (11). □

Example 6. Consider the LFSA S_3 in Fig. 8(a). We compute part of the concurrent composition $\mathrm{CC_A}(S_{3\mathrm{f}}, S_{3\mathrm{n}})$ as in Fig. 8(b). In $\mathrm{CC_A}(S_{3\mathrm{f}}, S_{3\mathrm{n}})$, one sees a run $(q_0, q_0) \xrightarrow{(e_1,e_1)} (q_1, q_2) \xrightarrow{(e_2,e_2)} (q_3, q_4) \xrightarrow{(f,\epsilon)} (q_5, q_4) \xrightarrow{(u,\epsilon)} (q_5, q_4)$, which satisfies (11). Then by Theorem 4.2, S_3 is not $\{f\}$-diagnosable.

The definition of diagnosability as in Definition 10 is in the form of ∃∀, which is similar to the definition of ω-strong detectability as in Definition 4 which is also in the form of ∃∀. As mentioned in Section 3, for strong versions of detectability, ∃∀ (i.e., ω-strong detectability) is strictly stronger than ∀∃ (i.e., ω-eventual strong detectability) (see Example 5). However, for diagnosability, ∃∀ is equivalent to ∀∃, where ∀∃ is formulated as follows:

$$(\forall s \in L(S) \cap E^* E_f)(\exists k_s \in \mathbb{N})(\forall s' : ss' \in L(S)) \tag{12a}$$
$$[(|s'| > k_s) \implies \mathbf{D}], \tag{12b}$$

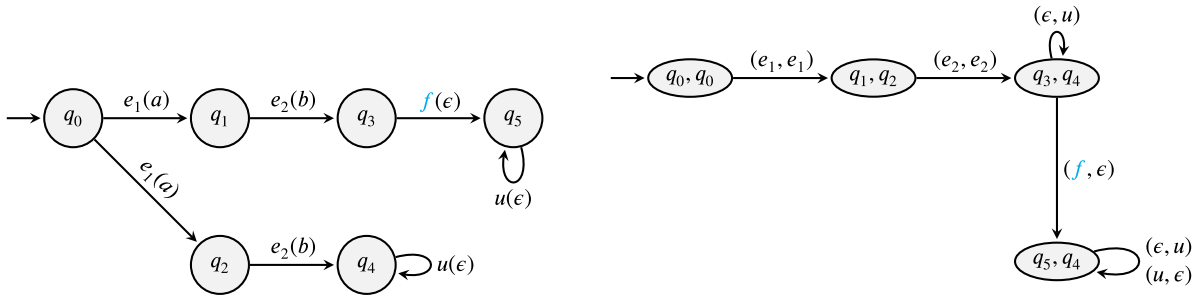
where **D** is defined in (9). The negation of (12) is as follows:

$$(\exists s \in L(S) \cap E^* E_f)(\forall k \in \mathbb{N}) \tag{13a}$$
$$(\exists s'_k : ss'_k \in L(S))(\exists s''_k \in L(S)) \tag{13b}$$
$$[(|s'_k| > k) \wedge (\ell(ss'_k) = \ell(s''_k)) \wedge (E_f \notin s''_k)]. \tag{13c}$$

Because of the finiteness of the number of states of an LFSA S, S satisfies (13) if and only if in $\mathrm{CC_A}(S_\mathrm{f}, S_\mathrm{n})$, there exists a run (10) satisfying (11). Hence ∃∀ is equivalent to ∀∃. From this conclusion, one can see that the strong versions of detectability are more complex than the two versions of diagnosability, because for the strong versions of detectability, ∃∀ (i.e., ω-strong detectability) is not equivalent to ∀∃ (i.e., ω-eventual strong detectability). Note that for diagnosability, the equivalence of ∀∃ and ∃∀ was also proven in Cabasino et al. (2012, Proposition 5.3) based on the assumption that after each faulty event occurs, the ending state is live.

5. Predictability

Different from diagnosability, *predictability* describes whether one can use an observed output sequence to be sure some faulty event will be certain to occur.



(a) LFSA S_3. (b) Part of $\mathrm{CC}_{\mathrm{A}}(S_{3\mathrm{f}}, S_{3\mathrm{n}})$.

Fig. 8. LFSA S_3 (left) and part of concurrent composition $\mathrm{CC}_{\mathrm{A}}(S_{3\mathrm{f}}, S_{3\mathrm{n}})$ (right), where only event f is faulty.

Table 3
Time complexity upper bounds on verifying predictability of LFSAs using different methods, where "¬" is short for "the negation of", S denotes an LFSA, "d.d." is short for "deadlock-free (i.e., live) and divergence-free", $|Q|$ and $|E|$ denote the numbers of states and events of an LFSA, respectively. In order not to depend on the two assumptions of deadlock-freeness and divergence-freeness, the square on $|E|$ is indispensable. The independence of the two assumptions was derived from verifying the negation of predictability. In a verifier, the states contain 4 components, where 2 of them are states of S, the other 2 are Y/N. While in a concurrent composition, the states are pairs of states of S. For deterministic S, the above complexity upper bounds degenerate by reducing 16 and $|Q|^4$ to their positive square roots.

	predictability		¬ predictability
d.d. S	$O(16\|Q\|^4\|E\|)$ (Genc & Lafortune, 2009) verifier	S	$O(\|Q\|^4\|E\|^2)$ (Zhang, 2021) concurrent composition

Definition 11 (*Predictability*). *(Genc & Lafortune, 2009)* Consider an LFSA S (1) and a subset $E_f \subset E$ of faulty events. S is called E_f-*predictable* if

$$(\exists k \in \mathbb{N})(\forall s \in L(S) \cap E^* E_f)$$
$$(\exists s' \sqsubset s : E_f \notin s')(\forall uv \in L(S))$$
$$[((\ell(s') = \ell(u)) \wedge (E_f \notin u) \wedge (|v| > k))$$
$$\Longrightarrow (E_f \in v)].$$

The same as in Sampath et al. (1995), predictability was defined on deterministic LFSAs that are deadlock-free and divergence-free. The verifier method proposed in Sampath et al. (1995) used for verifying predictability also depends on the two assumptions of deadlock-freeness and divergence-freeness. See Table 3 as a summary. Intuitively, if S is E_f-predictable, then once a faulty event will definitely occur, then before any faulty event occurs, one can be sure that after a common time delay (representing the number of occurrences of events, e.g., k), all generated finite traces with the same observation without any faulty event must be continued by a trace containing a faulty event, so as to raise an alarm to definite occurrence of some faulty event.

In order to verify E_f-predictability of S, we use the self-composition $\mathrm{CC}_{\mathrm{A}}(S_{\mathrm{n}}, S_{\mathrm{n}})$ of the normal subautomaton S_{n}. $\mathrm{CC}_{\mathrm{A}}(S_{\mathrm{n}}, S_{\mathrm{n}})$ is also computed similarly as in Definition 2.

Theorem 5.1 (*Zhang, 2021*). *Consider an LFSA S (1) and a subset $E_f \subset E$ of faulty events. S is not E_f-predictable if and only if in $\mathrm{CC}_{\mathrm{A}}(S_{\mathrm{n}}, S_{\mathrm{n}})$, there exists a run*

$$q'_0 \xrightarrow{s'_1} q'_1 \tag{14}$$

such that

$$q'_0 \in Q'_0; \tag{15a}$$

$$(q'_1(L), e_f, q) \in \delta \text{ for some } e_f \in E_f \text{ and } q \in Q; \tag{15b}$$

in S_{n}, there is a transition cycle

reachable from $q'_1(R)$. (15c)

Proof. By definition, S is not E_f-predictable if and only if

$$(\forall k \in \mathbb{N})(\exists s_k \in L(S) \cap E^* E_f)$$
$$(\forall s'_k \sqsubset s_k : E_f \notin s'_k)(\exists u_k v_k \in L(S))$$
$$[(\ell(s'_k) = \ell(u_k)) \wedge (E_f \notin u_k v_k) \wedge (|v_k| > k)].$$

Choose sufficiently large k, by the finiteness of the number of states of S and the pigeonhole principle, S is not E_f-predictable if and only if in $\mathrm{CC}_{\mathrm{A}}(S_{\mathrm{n}}, S_{\mathrm{n}})$, there exists a run (14) satisfying (15). □

Example 7. Reconsider the LFSA S_3 in Fig. 8(a). Part of the concurrent composition $\mathrm{CC}_{\mathrm{A}}(S_{3\mathrm{n}}, S_{3\mathrm{n}})$ can be obtained from Fig. 8(b) by removing the transition with event (f, ϵ). Then in $\mathrm{CC}_{\mathrm{A}}(S_{3\mathrm{n}}, S_{3\mathrm{n}})$, one sees a run $(q_0, q_0) \xrightarrow{(e_1,e_1)} (q_1, q_2) \xrightarrow{(e_2,e_2)} (q_3, q_4)$, which satisfies (15). Then by Theorem 5.1, S_3 is not $\{f\}$-predictable.

One can see that although the notion of predictability in Definition 11 itself is rather complex, its negation shown in Theorem 5.1 based on concurrent composition is rather simple.

6. Standard opacity

Opacity is a concealment-based (confidentiality) property which was first proposed by Mazaré (2004). Opacity describes whether the visit of *secrets* in a system could be forbidden from being leaked to an external *intruder*, given that the intruder knows complete knowledge of the system's structure but can only see generated outputs. It has been widely used to describe all kinds of scenarios in cyber security/privacy problems such as the dinning cryptographers problem (Chaum, 1988), chemical engineering and voting systems (Bryans et al., 2008), encryption using pseudo random generators and tracking of mobile agents in sensor networks (Saboori, 2010), ensuring privacy in location-based services (Wu, 2014), the indoor location privacy problem using obfuscation (Góes et al., 2018; Wu et al., 2016, 2018).

Bryans et al. (2008) proposed a general run-based opacity framework in labeled transition systems (which contain LFSAs, labeled Petri nets, etc., as subclasses), where such a system is opaque if for every secret run, there exists a non-secret run such that the two runs produce the same observation. Later on, two special types of secrets are studied:

subsets of event sequences (aka traces) and subsets of states. According to the two types of secrets, opacity is classified into language-based opacity and state-based opacity. The former refers to for every secret generated trace, there is a non-secret generated trace such that they produce the same observation; the latter refers to whenever a run passes through a secret state at some instant, there exists another run that does not pass any secret state at the same instant such that the two runs produce the same observation before and after the time instant. Language-based opacity is more involved, because it is already undecidable for LFSAs which contain no observable events (Bryans et al., 2008); particularly, when secret languages and non-secret languages are regular, language-based opacity is decidable in exponential time (Lin, 2011). State-based opacity is relatively simpler. When the time instant of visiting secret states is specified as the initial time, the current time, any past time, and at most K observational steps prior to the current time, the notions of state-based opacity can be formulated as *initial-state opacity* (ISO) (Saboori & Hadjicostis, 2013), *current-state opacity* (CSO) (Cassez et al., 2009), *infinite-step opacity* (InfSO) (Saboori & Hadjicostis, 2012), and *K-step opacity* (KSO) (Saboori & Hadjicostis, 2009), respectively. The problems of verifying the four types of state-based opacity are PSPACE-complete in LFSAs (Cassez et al., 2009; Saboori & Hadjicostis, 2012, 2013), the four properties and the special case of language-based opacity studied in Lin (2011) are polynomially reducible to each other (Balun & Masopust, 2021, 2022a; Wu & Lafortune, 2013).

Next, we show a concurrent-composition method to verify the four properties of state-based opacity. One can directly use an observer to verify CSO (Cassez et al., 2009; Saboori & Hadjicostis, 2007) and directly use a reverse observer to verify ISO (Wu & Lafortune, 2013). The verification methods in Cassez et al. (2009), Saboori and Hadjicostis (2007), Wu and Lafortune (2013) are currently the most efficient methods for verifying CSO and ISO. However, verifying InfSO and KSO are more difficult, currently one cannot see any possibility of directly using an observer or a reverse observer to do their verification. The concurrent-composition method to be shown to verify InfSO and KSO is more efficient than the initial-state-estimator method (Saboori & Hadjicostis, 2012), the two-way-observer method (i.e., the alternating product of an observer and a reverse observer) (Yin & Lafortune, 2017), and the method based on nonsecret specification automata and observer (Wintenberg et al., 2022). Hence the concurrent-composition method is currently the most efficient.

Definition 12 (*ISO*). *(Saboori & Hadjicostis, 2013)* Consider an LFSA S (1) and a subset $Q_S \subset Q$ of secret states. S is called *initial-state opaque* (ISO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0 \cap Q_S$, there exists a run $q_0' \xrightarrow{s'} q'$ such that $q_0' \in Q_0 \setminus Q_S$ and $\ell(s) = \ell(s')$.

From now on, ISO is short for "initial-state opacity" or "initial-state opaque" adapted to context. Analogous for CSO, InfSO, and KSO.

Intuitively, if an LFSA is ISO, then an external intruder cannot be sure whether the initial state is secret by observing generated label sequences.

Definition 13 (*CSO*). *(Cassez et al., 2009)* Consider an LFSA S (1) and a subset $Q_S \subset Q$ of secret states. S is called *current-state opaque* (CSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0$ and $q \in Q_S$, there exists a run $q_0' \xrightarrow{s'} q'$ such that $q_0' \in Q_0$, $q' \in Q \setminus Q_S$, and $\ell(s) = \ell(s')$.

If an LFSA is CSO, then an external intruder cannot be sure whether the current state is secret by observing generated label sequences.

Definition 14 (*InfSO*). *(Saboori & Hadjicostis, 2012)* Consider an LFSA S (1) and a subset $Q_S \subset Q$ of secret states. S is called *infinite-step opaque* (InfSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2$ with $q_0 \in Q_0$ and $q_1 \in Q_S$, there exists a run $q_0' \xrightarrow{s_1'} q_1' \xrightarrow{s_2'} q_2'$ such that $q_0' \in Q_0$, $q_1' \in Q \setminus Q_S$, $\ell(s_1) = \ell(s_1')$, and $\ell(s_2) = \ell(s_2')$.

Definition 15 (KSO). *(Saboori & Hadjicostis, 2009)* Consider an LFSA S (1), a subset $Q_S \subset Q$ of secret states, and a positive integer K. S is called *K-step opaque* (KSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2$ with $q_0 \in Q_0$, $q_1 \in Q_S$, and $|\ell(s_2)| \leq K$, there exists a run $q_0' \xrightarrow{s_1'} q_1' \xrightarrow{s_2'} q_2'$ such that $q_0' \in Q_0$, $q_1' \in Q \setminus Q_S$, $\ell(s_1) = \ell(s_1')$, and $\ell(s_2) = \ell(s_2')$.

If an LFSA is InfSO (KSO), then an external intruder cannot be sure whether any past state (at most K observational steps prior to the current time) is secret by observing generated label sequences.

In order to verify CSO, the notion of observer (Definition 3) is enough.

Theorem 6.1 (*Cassez et al., 2009; Saboori & Hadjicostis, 2007*). *An LFSA S (1) is CSO with respect to $Q_S \subset Q$ if and only if for every nonempty state x reachable in S_{obs}, $x \not\subset Q_S$.*

Example 8. Reconsider the LFSA S_2 in Fig. 2 and its observer $S_{2\,\text{obs}}$ in Fig. 3. By Theorem 6.1, S_2 is CSO with respect to $\{q_2\}$.

We next use the concurrent composition

$$\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}}) = (Q', E', \delta', Q_0', \Sigma, \ell') \tag{16}$$

to verify state-based opacity, where S_ε is obtained from S by changing each transition $q \xrightarrow{e} q'$ to $q \xrightarrow{\ell(e)} q'$ if $e \in E_o$, to $q \xrightarrow{\varepsilon} q'$ if $e \in E_{uo}$, and replacing the labeling function of S by the map $\hat{\ell}$ on $\ell(E_o) \cup \{\varepsilon\}$ satisfying that $\hat{\ell}|_{\ell(E_o)}$ (the restriction of $\hat{\ell}$ to $\ell(E_o)$) is the identity map and $\hat{\ell}(\varepsilon) = \varepsilon$; $S^\varepsilon_{\text{obs}}$ is obtained from S_{obs} by adding an additional event ε and the labeling function $\hat{\ell}$ of S_ε. In this particular case, $\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}})$ is almost the same as the parallel composition of S_ε and $S^\varepsilon_{\text{obs}}$ in Cassandras and Lafortune (2008, Page 80), because the observer $S^\varepsilon_{\text{obs}}$ as one component of the concurrent composition does not contain unobservable events and the events of $S^\varepsilon_{\text{obs}}$ are also events of S_ε. For more general cases, the concurrent composition of two LFSAs remarkably differs from their parallel composition. See Appendix.

The size of $\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}})$ is $O(|Q|2^{|Q|}(|\ell(E_o)||Q| + |E_{uo}||Q|)) = O(|Q|^2 2^{|Q|}|E|)$. The time consumption of computing $\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}})$ is also $O(|Q|^2 2^{|Q|}|E|)$ after S_ε and $S^\varepsilon_{\text{obs}}$ have been computed.

Theorem 6.2. *An LFSA S (1) is ISO with respect to $Q_S \subset Q$ if and only if (1) $Q_0 \neq \emptyset \implies Q_0 \not\subset Q_S$ and (2) for every $q_0 \in Q_0 \cap Q_S$, in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}})$, all states reachable from $(q_0, \text{UR}(Q_0 \setminus Q_S))$ are of the form $(-, x)$ with $x \neq \emptyset$.*

Theorem 6.3 (*Tong et al., 2018*). *An LFSA S (1) is CSO with respect to $Q_S \subset Q$ if and only if for every state (q, x) reachable in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}})$, if q is secret then $x \cap (Q \setminus Q_S) \neq \emptyset$.*

Theorem 6.4 (*Balun & Masopust, 2022b*). *An LFSA S (1) is InfSO with respect to $Q_S \subset Q$ if and only if for every nonempty state x reachable in S_{obs}, one has $x \not\subset Q_S$ and for every $q \in x \cap Q_S$, in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}})$, all states reachable from $(q, x \setminus Q_S)$ are of the form $(-, x')$ with $x' \neq \emptyset$.*

Theorem 6.5 (*Balun & Masopust, 2022b*). *Let K be a positive integer. An LFSA S (1) is KSO with respect to $Q_S \subset Q$ if and only if for every nonempty state x reachable in S_{obs}, one has $x \not\subset Q_S$ and for every $q \in x \cap Q_S$, in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\varepsilon_{\text{obs}})$, for every run $(q, x \setminus Q_S) \xrightarrow{s'} (q', x')$ with $|\ell'(s')| \leq K$, $x' \neq \emptyset$.*

Theorems 6.1, 6.2, 6.3, 6.4, and 6.5 directly follow from definition. By definition, one directly sees the following corollaries. If an LFSA S is KSO (with respect to $Q_S \subset Q$), then it is K'SO for any $K' < K$. Conversely, if S is not KSO with $K > 2^{|Q|} - 2$, then it is not K'SO for some $K' \leq 2^{|Q|} - 2$, because S_{obs} has at most $2^{|Q|} - 1$ nonempty

Table 4
Time complexity upper bounds on verifying various notions of standard state-based opacity of LFSAs using different methods, where $|Q|$, $|E|$, and $|\ell(E_o)|$ denote the numbers of states, events, and distinct labels of the observable states of an LFSA, respectively. The algorithm in Theorem 6.2 (resp., (Tong et al., 2018)) for verifying initial-state opacity (resp., current-state opacity) is a little less efficient than the algorithm in Wu and Lafortune (2013) (resp., Cassez et al., 2009), because in the former, additional time consumption is needed to compute the concurrent composition $\mathrm{CC}_{\mathrm{A}}(S_{\varepsilon}, S^{\varepsilon}_{\mathrm{obs}})$.

method	initial-state opacity	current-state opacity	infinite-step opacity	K-step opacity
reverse observer	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Wu & Lafortune, 2013)			
observer		$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Saboori & Hadjicostis, 2007) (Cassez et al., 2009)		
concurrent composition + observer	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ Theorem 6.2	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Tong et al., 2018, called I-observer)	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Balun & Masopust, 2021, called product automaton)	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Balun & Masopust, 2021, called product automaton)
initial-state estimator	$O(2^{\|Q\|^2}\|\ell(E_o)\|)$ (Saboori & Hadjicostis, 2013)		$O(2^{\|Q\|+\|Q\|^2}\|\ell(E_o)\|)$ (Saboori & Hadjicostis, 2012)	$O((\|\ell(E_o)\|+1)^{\min\{K,2^{\|Q\|^2}-1\}}2^{\|Q\|}\|\ell(E_o)\|)$ (Saboori & Hadjicostis, 2009)
two-way observer			$O(2^{\|Q\|+1}\|Q\|^2\|\ell(E_o)\|\|E\|+4^{\|Q\|}\|\ell(E_o)\|)$ (Yin & Lafortune, 2017)	$O(2^{\|Q\|+1}\|Q\|^2\|\ell(E_o)\|\|E\|+\min\{2^{\|Q\|},\|\ell(E_o)\|^K\}2^{\|Q\|}\|\ell(E_o)\|)$ (Yin & Lafortune, 2017)
nonsecret specification automaton + observer	$O(3\cdot 2^{3\|Q\|+2}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Wintenberg et al., 2022)	$O(2^{2\|Q\|+3}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Wintenberg et al., 2022)	$O((2^{\|Q\|}+3)2^{(2^{\|Q\|}+3)\|Q\|+2}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Wintenberg et al., 2022)	$O((K+3)2^{(K+3)\|Q\|+2}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Wintenberg et al., 2022)

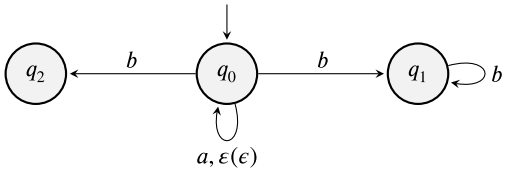


Fig. 9. LFSA $S_{2\varepsilon}$ corresponding to the LFSA S_2 in Fig. 2.

states; then it is not $(2^{|Q|}-2)$SO. Hence the verification of KSO based on Theorem 6.5 does not depend on K if $K > 2^{|Q|}-2$. The verification algorithms shown in Theorems 6.1, 6.2, 6.3, 6.4, and 6.5 all run in time $O(2^{|Q|}|Q|^2|\ell(E_o)||E|)$. The above upper bound $2^{|Q|}-2$ on K is obtained directly from definition. The upper bound was also obtained in Yin and Lafortune (2017) but based on the two-way observer. The upper bound for K obtained in Saboori and Hadjicostis (2009) is $2^{|Q|^2}-2$. Compared with the concurrent-composition method, the relative inefficiency of the two-way observer method (Yin & Lafortune, 2017) comes from computing a reverse observer (with the same complexity as computing an observer) and the alternating product (i.e., the so-called two-way observer, with time complexity $O(4^{|Q|}|\ell(E_o)|)$) of the observer and the reverse observer. The verification algorithms obtained in Saboori and Hadjicostis (2009, 2012), Wintenberg et al. (2022) have even higher complexity. See Table 4 as a summary.

Corollary 6.6 (*Yin & Lafortune, 2017*). *Let K be a positive integer. An LFSA S (1) is KSO with respect to $Q_S \subset Q$ if and only if it is* $\min\{K, 2^{|Q|}-2\}$*SO with respect to Q_S.*

Corollary 6.7 (*Yin & Lafortune, 2017*). *An LFSA S (1) is InfSO with respect to $Q_S \subset Q$ if and only if it is KSO with respect to Q_S with $K > 2^{|Q|}-2$.*

Example 9. Reconsider the LFSA S_2 in Fig. 2 and its observer $S_{2\,\mathrm{obs}}$ in Fig. 3. The corresponding $S_{2\varepsilon}$ is shown in Fig. 9. The concurrent composition $\mathrm{CC}_{\mathrm{A}}(S_{2\varepsilon}, S^{\varepsilon}_{2\,\mathrm{obs}})$ is shown in Fig. 10.

By Theorem 6.2, S_2 is not ISO with respect to $\{q_0\}$, because q_0 is the unique initial state. In addition, one has S_2 is InfSO with respect to $\{q_2\}$ by Theorem 6.4, because the unique reachable state of $S_{2\,\mathrm{obs}}$ containing q_2 is $\{q_1, q_2\}$ and in $\mathrm{CC}_{\mathrm{A}}(S_{2\varepsilon}, S^{\varepsilon}_{2\,\mathrm{obs}})$, there is no state reachable from $(q_2, \{q_1\})$ of the form $(-, \emptyset)$. By the reachable state $\{q_1\}$ of $S_{2\,\mathrm{obs}}$, one sees S_2 is not InfSO with respect to $\{q_1\}$, which can also be seen from the fact that $\{q_1, q_2\}$ is reachable in $S_{2\,\mathrm{obs}}$ and in $\mathrm{CC}_{\mathrm{A}}(S_{2\varepsilon}, S^{\varepsilon}_{2\,\mathrm{obs}})$, the state $(q_1, \emptyset)$ is reachable from $(q_1, \{q_2\})$.

7. Strong opacity

In Section 6, variants of notions of opacity were shown to describe the ability of an LFSA to forbid the visit to its secret states from being leaked to an external intruder. Sometimes, such "standard" opacity is not sufficiently strong, e.g., in some InfSO LFSA, when observing a generated label sequence, one can be sure that some secret state must have been visited, although cannot be sure of the exact visit instant of time. Consider the LFSA S_4 in Fig. 11. S_4 is InfSO with respect to $\{q_2, q_3\}$. When observing a, one can be sure that at least one secret state has been visited, in detail, if $q_1 \xrightarrow{a} q_2$ was generated then q_2 was visited, if $q_3 \xrightarrow{a} q_4$ was generated then q_3 was visited. This leads to a "strong version" of InfSO which guarantees that an intruder cannot be sure whether the current state is secret, and can also guarantee that the intruder cannot be sure whether some secret state has been visited. Analogously, the other three standard versions of opacity studied in Section 6 could also be reformulated as their strong versions.

In order to define strong versions of state-based opacity, we define a *non-secret run* of an LFSA by a run containing no secret states.

Definition 16 (*SISO*). (*Han et al., 2023*) Consider an LFSA S (1) and a subset $Q_S \subset Q$ of secret states. S is called *strongly initial-state opaque* (SISO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0 \cap Q_S$, there exists a *non-secret* run $q'_0 \xrightarrow{s'} q'$ such that $q'_0 \in Q_0 \setminus Q_S$ and $\ell(s) = \ell(s')$.

If an LFSA is SISO, then an external intruder cannot be sure whether the initial state is secret and cannot be sure whether some secret state has been visited either, by observing generated label sequences.

Definition 17 (*SCSO*). (*Han et al., 2023*) Consider an LFSA S (1) and a subset $Q_S \subset Q$ of secret states. S is called *strongly current-state opaque* (SCSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0$ and $q \in Q_S$, there exists a *non-secret* run $q'_0 \xrightarrow{s'} q'$ such that $q'_0 \in Q_0$ and $\ell(s) = \ell(s')$.

If an LFSA is SCSO, then an external intruder cannot be sure whether the current state is secret and cannot be whether some secret state has been visited either, by observing generated label sequences.

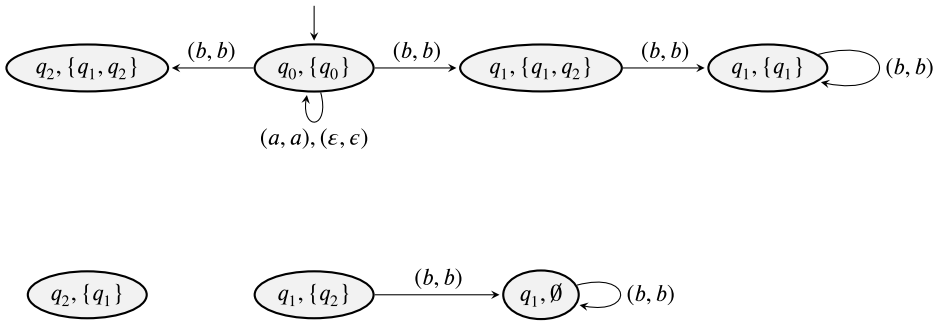


Fig. 10. Part of $\mathrm{CC}_{\mathrm{A}}(S_{2\epsilon}, S^{\epsilon}_{2\,\mathrm{obs}})$ corresponding to the LFSA S_2 in Fig. 2.



Fig. 11. LSFA S_4, where $\ell(a) = a$, q_2 and q_3 are secret, q_1 and q_4 are not.

Definition 18 (*SInfSO*). *(Han et al., 2023; Ma et al., 2021)* Consider an LFSA S (1) and a subset $Q_S \subset Q$ of secret states. S is called *strongly infinite-step opaque* (SInfSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2$ with $q_0 \in Q_0$ and $q_1 \in Q_S$, there exists a *non-secret* run $q'_0 \xrightarrow{s'_1} q'_1$ such that $q'_0 \in Q_0$ and $\ell(s_1 s_2) = \ell(s'_1)$.

Definition 18 coincides with (Han et al., 2023, Definition 5.1). Particularly for deterministic LFSAs with $\ell|_{E_o}$ injective, Definition 18 coincides with (Ma et al., 2021, Definition 3.2).

Definition 19 (*SKSO*). Consider an LFSA S (1), a subset $Q_S \subset Q$ of secret states, and a positive integer K. S is called *strongly K-step opaque* (SKSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2$ with $q_0 \in Q_0$, $q_1 \in Q_S$, and $|\ell(s_2)| \leq K$, there exists a *non-secret* run $q'_0 \xrightarrow{s'_1} q'_1$ such that $q'_0 \in Q_0$ and $\ell(s_1 s_2) = \ell(s'_1)$.

From now on, SISO is short for "strong initial-state opacity" or "strongly initial-state opaque" adapted to context. Analogous for SCSO, SInfSO, and SKSO.

If an LFSA is SInfSO (SKSO), then an external intruder cannot be sure whether any past state (at most K observational steps prior to the current time) is secret and cannot be sure whether some secret state has been visited either, by observing generated label sequences.

The SKSO is slightly stronger than the strong K-step opacity previously studied in Ma et al. (2021) (see Definition 3.1 therein) and the K-step strong opacity previously studied in Falcone and Marchand (2015) (see Definition 4 therein). In these two papers, deterministic LFSAs with $\ell|_{E_o}$ injective were studied.

Definition 20 (*KSSO*).*(Falcone & Marchand, 2015)* Consider a deterministic LFSA S (1) with $\ell|_{E_o}$ injective, a subset $Q_S \subset Q$ of secret states, and a positive integer K. S is called *K-step strongly opaque* (*K*SSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s_1} q_1$ with q_0 the initial state, there exists a run $q_0 \xrightarrow{s'_1} q'_1$ such that $\ell(s_1) = \ell(s'_1)$ and for all $s''_1 \sqsubset s'_1$ with $|\ell(s'_1)| - |\ell(s''_1)| \leq K$, $\delta(q_0, s''_1) \subset Q \setminus Q_S$.

Definition 21 (*KSSO*).*(Ma et al., 2021)* Consider a deterministic LFSA S (1) with $\ell|_{E_o}$ injective, a subset $Q_S \subset Q$ of secret states, and a positive integer K. S is called *strongly K-step opaque* (*K*SSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2$ with q_0 the initial state, $q_1 \in Q_S$, and $|\ell(s_2)| \leq K$, there exists a run $q_0 \xrightarrow{s'_1} q'_1$ such that $\ell(s_1 s_2) = \ell(s'_1)$ and for all $s''_1 \sqsubset s'_1$ with $|\ell(s'_1)| - |\ell(s''_1)| \leq K$, $\delta(q_0, s''_1) \subset Q \setminus Q_S$.

Proposition 7.1. *Definition 20 is equivalent to Definition 21.*

Proof. It is trivial to see Definition 20 is stronger than Definition 21.

To see the converse, assume a deterministic LFSA S with $\ell|_{E_o}$ injective that satisfies Definition 21. Without loss of generality, consider a run $q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2$ with $q_0 \in Q_0$ that contains at least one secret state. Let q_1 be the last secret state. Assume $|\ell(s_2)| \leq K$ without loss of generality, then Definition 21 implies there is a run as the run $q'_0 \xrightarrow{s'_1} q'_1$ in Definition 20. □

We generalize the definition of *K*SSO to general LFSAs.

Definition 22 (*KSSO*). Consider an LFSA S (1), a subset $Q_S \subset Q$ of secret states, and a positive integer K. S is called *K-step strongly opaque* (*K*SSO) *with respect to* Q_S if for every run $q_0 \xrightarrow{s_1} q_1$ with $q_0 \in Q_0$, there exists a run $q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{s'_2} q'_2$ such that $q'_0 \in Q_0$, $\ell(s_1) = \ell(s'_1 s'_2)$, $s'_1 \neq \epsilon \implies \mathrm{last}(s'_1) \in E_o$, $|\ell(s'_1 s'_2)| > K \implies |\ell(s'_2)| = K$, $|\ell(s'_1 s'_2)| \leq K \implies s'_1 = \epsilon$, and $q'_1 \xrightarrow{s'_2} q'_2$ is non-secret.

In Definition 22, in the run $q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{s'_2} q'_2$, the possibly longest suffix with K observational length is non-secret, which can forbid the visit to a secret state at most K observational steps prior to the current time. It is straightforward to see that Definition 22 coincides with Definitions 20 and 21 when the LFSA under consideration is deterministic and $\ell|_{E_o}$ is injective.

Not so strong as SKSO, *K*SSO implies that in the last at most K observational steps, one cannot be sure whether some secret state has been visited.

Next we use the concurrent-composition structure to do verification for these strong versions of state-based opacity, where the derived verification algorithms are more efficient than the K/Inf-step recognizer method proposed by Ma et al. (2021). Recall that the automata studied in Ma et al. (2021) are deterministic LFSAs with $\ell|_{E_o}$ injective. The time consumption of computing an Inf-step recognizer (used for verify SInfSO) is bounded (from above) by $\sum_{Q' \subset Q \times \{0,\infty\}} \sum_{|E_o|} (|Q'| + |Q||E_{uo}|) = |E_o||Q|4^{|Q|}(1 + |E_{uo}|)$, the time consumption of computing a K-step recognizer (used for verify the strong K-step opacity in Ma et al. (2021)) is bounded by $\sum_{Q' \subset Q \times \{0,\ldots,K+1\}} \sum_{|E_o|} (|Q'| + (K+2)|Q||E_{uo}|) = (K+2)|E_o||Q|2^{(K+2)|Q|-1}(1+2|E_{uo}|)$, for the deterministic LFSAs studied in Ma et al. (2021). For each K, one needs to compute the corresponding K-step recognizer. However, by our concurrent-composition method, one only needs to compute one concurrent composition, and then use it to verify SKSO and *K*SSO for any $K > 0$. The SKSO as in Definition 19 was not studied in Falcone and Marchand (2015), Ma et al. (2021).

Consider an LFSA $S = (Q, E, \delta, Q_0, \Sigma, \ell)$ and a subset $Q_S \subset Q$ of secret states, let

$$S_{\mathrm{dss}} = (Q_{\mathrm{dss}}, E_{\mathrm{dss}}, \delta_{\mathrm{dss}}, Q_{0\,\mathrm{dss}}, \Sigma, \ell_{\mathrm{dss}})$$

be the remainder of S by *deleting secret states* (dss) of S. Let

$$S_{\text{dss obs}} = (Q_{\text{dss obs}}, \ell_{\text{dss}}(E_{\text{dss}}) \setminus \{\epsilon\}, \delta_{\text{dss obs}}, q_{0\,\text{dss obs}})$$

be the observer of S_{dss}.

Similar to S_{obs}, the size of $S_{\text{dss obs}}$ is $O(2^{|Q|}|\ell(E_o)|)$, the time consumption of computing $S_{\text{dss obs}}$ is $O(2^{|Q|}|Q|^2|\ell(E_o)||E|)$. The size of $S_{\text{dss obs}}$ is slightly smaller than that of S_{obs}.

We will use the concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$ to verify the five strong versions of opacity. Unlike CSO and ISO, SCSO and SISO cannot be verified by directly using the notions of observer and reverse observer.

Similar to $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{obs}})$, the size of $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$ is $O(|Q|^2 2^{|Q|}|E|)$. The time consumption of computing $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$ is also $O(|Q|^2 2^{|Q|}|E|)$ after S_ε and $S^\epsilon_{\text{dss obs}}$ have been computed.

Theorem 7.2 (*Han et al., 2023*). *An LFSA S (1) is SCSO with respect to $Q_S \subset Q$ if and only if for every state (q, x) reachable in $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$, if $q \in Q_S$ then $x \neq \emptyset$.*

Theorem 7.3 (*Han et al., 2023*). *An LFSA S (1) is SISO with respect to $Q_S \subset Q$ if and only if (1) $Q_0 \neq \emptyset \implies Q_0 \not\subset Q_S$ and (2) for every $q_0 \in Q_0 \cap Q_S$, in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$, all states reachable from $(q_0, q_{0\,\text{dss obs}})$ are of the form $(-, x)$ with $x \neq \emptyset$.*

Theorem 7.4 (*Han et al., 2023*). *An LFSA S (1) is SInfSO with respect to $Q_S \subset Q$, if and only if, (i) for every state (q, x) reachable in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$, if $q \in Q_S$ then $x \neq \emptyset$ and all states reachable from (q, x) are of the form $(-, x')$ with $x' \neq \emptyset$, if and only if, (ii) all states (q'', x'') reachable in $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$ satisfy $x'' \neq \emptyset$.*

Proof. By definition, (i) is equivalent for S to be SInfSO with respect to Q_S.

(ii) $\implies$ (i): This trivially holds.

(i) $\implies$ (ii): Consider a state (q'', x'') reachable in $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$. If there is a run from some initial state of $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$ to (q'', x'') containing a state (q, x) with $q \in Q_S$, then by (i), one has $x'' \neq \emptyset$; otherwise one also has $x'' \neq \emptyset$ because $q'' \in x''$ by definition of $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$. □

Theorem 7.5. *Let K be a positive integer. An LFSA S (1) is SKSO with respect to $Q_S \subset Q$ if and only if for every state (q, x) reachable in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$, if $q \in Q_S$ then $x \neq \emptyset$ and for every run $(q, x) \xrightarrow{s'} (q', x')$ with $|\ell'(s')| \leq K$, $x' \neq \emptyset$.*

Similar to the standard KSO, there is also an upper bound on K for SKSO. The upper bound is $|Q|(2^{|Q\setminus Q_S|} - 1)$. Hence the verification of SKSO based on Theorem 7.5 does not depend on K if $K > |Q|(2^{|Q\setminus Q_S|} - 1)$. The verification algorithms shown in Theorems 7.2, 7.3, and 7.4 all run in time $O(2^{|Q|}|Q|^2|\ell(E_o)||E|)$. The verification algorithm shown in Theorem 7.5 runs in time $O(2^{|Q|}|Q|^2|\ell(E_o)||E| + \min\{K, |Q|2^{|Q\setminus Q_S|-1}\}2^{|Q|}|Q|^2|\ell(E_o)|)$.

Proposition 7.6. *Let K be a positive integer. An LFSA S (1) is SKSO with respect to $Q_S \subset Q$ if and only if it is S$\min\{K, |Q|(2^{|Q\setminus Q_S|}-1)\}$SO with respect to Q_S.*

Proof. By definition, if S is SKSO (respect to $Q_S \subset Q$), then S is SK′SO for any K' in $[\![0, K-1]\!]$.

Conversely, assume S is not SKSO, where $K > |Q|(2^{|Q\setminus Q_S|}-1)$. Then in S_ε there exists a run

$$q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2 \tag{17}$$

such that $q_0 \in Q_0$, $q_1 \in Q_S$, $|\bar{s}_2| = K$, and $\delta_{\text{dss obs}}(q_{0\,\text{dss obs}}, \bar{s}_1\bar{s}_2) = \emptyset$, $\bar{s}_i$ is obtained from s_i by removing all ϵ's, $i = 1, 2$. If $\delta_{\text{dss obs}}(q_{0\,\text{dss obs}}, \bar{s}_1) = \emptyset$ then S is not S0SO. Next we assume $\delta_{\text{dss obs}}(q_{0\,\text{dss obs}}, \bar{s}_1) \neq \emptyset$. Then by combining the run (17) and the run $q_{0\,\text{dss obs}} \xrightarrow{\bar{s}_1} x_1 \xrightarrow{\bar{s}_2} \emptyset$ $(x_1 \neq \emptyset)$, in concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$ we obtain a run

$$(q_0, q_{0\,\text{dss obs}}) \xrightarrow{s'_1} (q_1, x_1) \xrightarrow{s'_2} (q_2, \emptyset), \tag{18}$$

where $s'_i(L) = s_i$, $s'_i(R) = \bar{s}_i$, $i = 1, 2$. By definition, in a run of concurrent composition $\text{CC}_{\text{A}}(S_\varepsilon, S^\epsilon_{\text{dss obs}})$, if a state has it right component equal to $\emptyset$, then all its subsequent states have their right components equal to $\emptyset$. Choose a sub-run of (18) as

$$(q_1, x_1) \xrightarrow{s'_3} (q_3, x_3), \tag{19}$$

where $s'_3 \sqsubset s'_2$, (q_3, x_3) is the rightmost state of (18) whose right component is nonempty. If (19) contains more than $|Q|(2^{|Q\setminus Q_S|} - 1)$ states, then it must contain repetitive states. If (19) contains a cycle $\diamond \to \diamond$, where $\diamond$ denotes a state of (19), then replace the cycle by $\diamond$. Repeat this modification until there is no cycle in (19). Then the remainder of (19) has length at most $|Q|(2^{|Q\setminus Q_S|} - 2)$. Hence S is not S$|Q|(2^{|Q\setminus Q_S|} - 1)$SO. □

Corollary 7.7. *An LFSA S (1) is SInfSO with respect to $Q_S \subset Q$ if and only if it is SKSO with respect to Q_S and positive integer K with $K > |Q|(2^{|Q\setminus Q_S|} - 1)$.*

The procedure of verifying KSSO is more involved than that of verifying SKSO as in Theorem 7.5, because in KSSO as in Definition 20, the run $q'_0 \xrightarrow{s'_1} q'_1$ has more restrictions than the run $q'_0 \xrightarrow{s'_1} q'_1$ in Definition 19. In order to verify KSSO, we need a slight variant of the observer S_{obs} as in Definition 3:

$$\tilde{S}_{\text{obs}} = (Q_{\text{obs}}, \ell(E_o), \tilde{\delta}_{\text{obs}}, \tilde{q}_{0\,\text{obs}}), \tag{20}$$

where

1. for all $x \in Q_{\text{obs}}$ and $a \in \ell(E_o)$, $\tilde{\delta}_{\text{obs}}(x, a) = \bigcup_{q\in x} \bigcup_{\substack{e_a \in E_o \\ \ell(e_a)=a \\ s\in(E_{uo})^*}} \delta(q, se_a)$,
2. $\tilde{q}_{0\,\text{obs}} = Q_0$.

Comparing $\tilde{S}_{\text{obs}}$ with S_{obs}, one can see minor differences between them. First, their initial states are not necessarily the same. Second, they have different transition functions. For a state $x \in Q_{\text{obs}}$ and a label $a \in \ell(E_o)$, the next-step state $\delta_{\text{obs}}(x, a)$ in S_{obs} is the set of states of Q reachable from the states of x via an observable event with label a followed by a sequence of unobservable events, while the next-step state $\tilde{\delta}_{\text{obs}}(x, a)$ in $\tilde{S}_{\text{obs}}$ is the set of states of Q reachable from the states of x via a sequence of unobservable events followed by an observable event with label a.

Let $\tilde{S}^\epsilon_{\text{dss obs}}$ be obtained from S_{dss} by firstly computing the variant observer $\tilde{S}_{\text{dss obs}}$ of S_{dss} as in (20) and secondly adding an event ϵ into $\tilde{S}_{\text{dss obs}}$ as in (16).

Theorem 7.8. *Consider an LFSA S (1), a subset $Q_S \subset Q$ of secret states, and a positive integer K. S is KSSO with respect to $Q_S \subset Q$ (with respect to Definition 22) if and only if in $\tilde{S}_{\text{obs}}$, for every reachable, nonempty state x and every $q \in x$, in $\text{CC}_{\text{A}}(S^\epsilon, \tilde{S}^\epsilon_{\text{dss obs}})$, for every run $(q, x \setminus Q_S) \xrightarrow{s'} (q', x')$ with $|\ell(s')| \leq K$, $x' \neq \emptyset$.*

Proof. "if":

Assume that the condition after "if and only if" holds. Without loss of generality, we consider a run

$$q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} q_2 \xrightarrow{s_3} q_3, \tag{21}$$

where $q_0 \in Q_0$, $q_2 \in Q_S$, $|\ell(s_3)| \leq K$. If $|\ell(s_1s_2s_3)| > K$, we let $|\ell(s_2s_3)| = K$ and $\text{last}(s_1) \in E_o$; otherwise, let $s_1 = \epsilon$, then q_0 coincides with q_1.

Then by features of $\tilde{S}_{\text{obs}}$ and $\text{CC}_{\text{A}}(S^\epsilon, \tilde{S}^\epsilon_{\text{dss obs}})$, there exists a run

$$q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{s'_3} q'_3 \tag{22}$$

Table 5
Time complexity upper bounds on verifying various notions of strong state-based opacity of LFSAs using different methods, where $|Q|$, $|E|$, $|E_{uo}|$, and $|\ell(E_o)|$ denote the numbers of states, events, unobservable events, and distinct labels of the observable states of an LFSA, respectively; Q_S denotes the set of secret states. In Falcone and Marchand (2015), Ma et al. (2021), deterministic LFSAs were studied.

method	strong initial-state opacity	strong current-state opacity	strong infinite-step opacity
concurrent composition + observer	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Han et al., 2023)	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Han et al., 2023)	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\|)$ (Han et al., 2023)
Inf-step recognizer			$O(4^{\|Q\|}\|Q\|\|\ell(E_o)\|(1+\|E_{uo}\|))$ (Ma et al., 2021)
method	strong K-step opacity	K-step strong opacity	
concurrent composition + observer	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\| + \min\{K, \|Q\|2^{\|Q\setminus Q_S\|-1}\}2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|)$ Theorem 7.5	$O(2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|\|E\| + \min\{K, \|Q\|2^{\|Q\setminus Q_S\|-1}\}2^{\|Q\|}\|Q\|^2\|\ell(E_o)\|)$ Theorem 7.8	
K-step recognizer	$O((K+2)2^{(K+2)\|Q\|-1}\|Q\|\|\ell(E_o)\|(1+2\|E_{uo}\|))$ (Ma et al., 2021)		
K-delay trajectory estimator		$O(2^{\|Q\|^{K+1}2^K}\|\ell(E_o)\|)$ (Falcone & Marchand, 2015)	

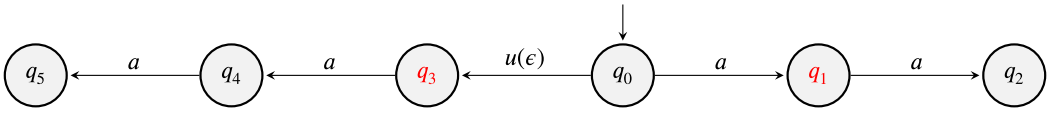


Fig. 12. LSFA S_5, where $\ell(u)=\epsilon$, $\ell(a)=a$, q_1 and q_3 are secret, the other states are not.

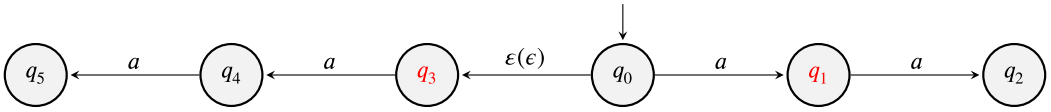


Fig. 13. $S_{5\varepsilon}$ corresponding to the LFSA S_5 in Fig. 12.

such that $q_0' \in Q_0$, $q_1' \xrightarrow{s_3'} q_3'$ is non-secret, $\text{last}(s_1') \in E_o$, $\ell(s_1)=\ell(s_1')$, $\ell(s_2s_3)=\ell(s_3')$, and $\text{last}(s_3') \in E_o$. That is, S is KSSO with respect to $Q_S \subset Q$. Note that $\{q_1, q_1'\} \subset \tilde{\delta}_{\text{obs}}(\tilde{q}_{0\,\text{obs}}, \ell(s_1'))$.

The "only if" part holds by definition. □

Similar to Proposition 7.6, the following result holds.

Proposition 7.9. *Consider an LFSA S (1), a subset $Q_S \subset Q$ of secret states, and a positive integer K. S is KSSO with respect to $Q_S \subset Q$ if and only if it is* $\min\{K, |Q|(2^{|Q\setminus Q_S|}-1)\}$*SSO with respect to Q_S.*

Remark 7.10. The highest time consumption in the verification algorithms shown in Theorems 6.1, 6.2, 6.3, 6.4, 6.5, 7.2, 7.3, 7.4, 7.5, and 7.8 comes from computing the corresponding observer. If the observer is not explicitly computed, then by using nondeterministic search, verification can be done in PSPACE. See Table 5 as a summary.

Example 10. Consider the LFSA S_5 in Fig. 12. We verify whether S_5 is InfSO or SInfSO with respect to $\{q_1, q_3\}$ by Theorems 6.4 and 7.4. By Theorem 6.4, we compute $S_{5\varepsilon}$, $S_{5\,\text{obs}}$, and $\text{CC}_{\text{A}}(S_{5\varepsilon}, S^{\varepsilon}_{5\,\text{obs}})$ as in Figs. 13, 14, and 15, respectively.

In observer $S_{5\,\text{obs}}$, the reachable states containing secret states are $\{q_0, q_3\}$ and $\{q_1, q_4\}$. In $\text{CC}_{\text{A}}(S_{5\varepsilon}, S^{\varepsilon}_{5\,\text{obs}})$, the states reachable from $(q_3, \{q_0\})$ and $(q_1, \{q_4\})$ all satisfy that their right components are nonempty. Then by Theorem 6.4, S_5 is InfSO with respect to $\{q_1, q_3\}$.

By Theorem 7.4, we compute $S_{5\,\text{dss}}$, $S_{5\,\text{dss obs}}$, and $\text{CC}_{\text{A}}(S_{5\varepsilon}, S^{\varepsilon}_{5\,\text{dss obs}})$ as in Figs. 16 and 17, respectively.

In $\text{CC}_{\text{A}}(S_{5\varepsilon}, S^{\varepsilon}_{5\,\text{dss obs}})$, there exist reachable states whose right components are equal to $\emptyset$, then by Theorem 7.4, S_5 is not SInfSO with respect to $\{q_1, q_3\}$.

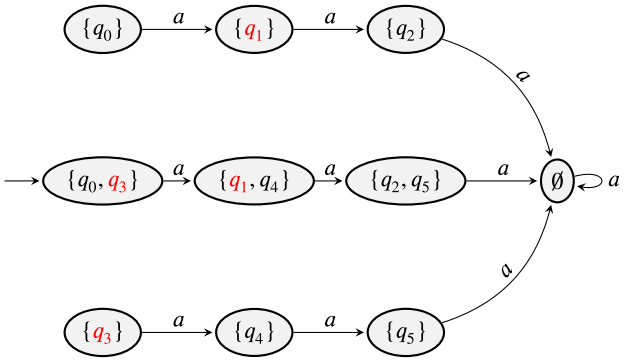


Fig. 14. Part of $S_{5\,\text{obs}}$ corresponding to the LFSA S_5 in Fig. 12.

8. Representations of inference-based properties and concealment-based properties in linear temporal logic

Zhao et al. (2022) conceptually transformed inference-based properties and concealment-based properties for deadlock-free and divergence-free LFSAs into a generalized version of linear temporal logic (LTL) — hyper LTL formulae, where hyper LTL formulae also admit infinite trace variables, but LTL formulae do not. Thus, these properties can be verified by using hyper LTL model checkers. There are advantages and disadvantages of making such an attempt. It is good to see that there is another possible way to verify these properties in a unified way. However, there are essential drawbacks in this way. The same as in a big amount of related publications, Zhao et al. (2022) made the two assumptions of deadlock-freeness and divergence-freeness, and verified the properties themselves instead of their negations, resulting in that the obtained results are still based

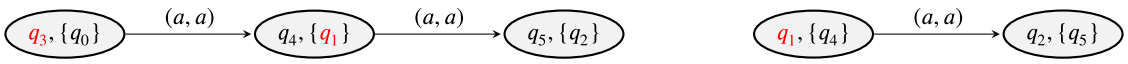
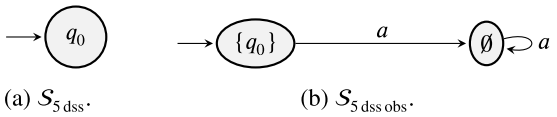


Fig. 15. Part of $\mathrm{CC_A}(S_{5\varepsilon}, S^{\varepsilon}_{5\,\mathrm{obs}})$ corresponding to the LFSA S_5 in Fig. 12.



(a) $S_{5\,\mathrm{dss}}$. (b) $S_{5\,\mathrm{dss\,obs}}$.

Fig. 16. $S_{5\,\mathrm{dss}}$ and $S_{5\,\mathrm{dss\,obs}}$ corresponding to the LFSA S_5 in Fig. 12.

on the two assumptions, because temporal logic model checking techniques usually apply to deadlock-free transition systems. Until now, there has been no modification technique that can make an LFSA deadlock-free and divergence-free and meanwhile preserve a property of interest except for only one modified notion of diagnosability (see Definition 1 and Remark 1 of Qiu and Kumar (2006), discussed in Remark 4.1). For example, the widely-used technique of adding a sink state to an LFSA to make it deadlock-free may not preserve a property of interest, e.g., this drawback applies to detectability, diagnosability (see Zhang, 2021, Remark 3.20), and predictability (see Zhang, 2021, Remark 3.27).

Next, we show that many of these properties can be equivalently transformed to LTL formulae, even based on no assumption, by using the concurrent-composition technique. Therefore, usually, it is not necessary to use the more advanced hyper LTL techniques. Moreover, since directly using the concurrent-composition technique has already provided the most efficient ways to do verification for these properties, it is not necessary to use LTL formulae either in the purpose of verification. Despite of this, LTL formulae show advantages, e.g., after using the concurrent composition to transform the negations of these properties to LTL formulae, one can use well known LTL model checkers to verify these properties. Finally, it is interesting to see what these properties look like after they are transformed to LTL formulae.

8.1. Strong detectability

In Zhao et al. (2022), the studied strong detectability (in Definition 5 therein) is equivalent to the $*$-strong detectability as shown in Definition 6 of the current paper. Under the two assumptions of deadlock-freeness and divergence-freeness, $*$-strong detectability was equivalently represented by using infinite generated label sequences as in Theorem 4 of Zhao et al. (2022), which is a hyper LTL formula. It can be seen that this representation is not necessary and sufficient without the two assumptions. See the following example.

Example 11. Reconsider LFSA $\bar{S}_2$ (Fig. 4(a)) that is obtained from the LFSA S_2 in Fig. 2(a) by removing the self-loop on state q_1. Then the self-composition $\mathrm{CC_A}(\bar{S}_2)$ can be obtained from Fig. 2(b) by removing the self-loop on (q_1, q_1). By Theorem 3.2, $\bar{S}_2$ is not $*$-strongly detectable. However, because the only possible infinite traces of $\bar{S}_2$ are of the form $q_0\left(\xrightarrow{e} q_0\right)^{\omega}$, where $e = e_1$ or e_2, $\bar{S}_2$ satisfies the equivalent condition for $*$-strong detectability shown in Theorem 4 of Zhao et al. (2022). Hence (Zhao et al., 2022, Theorem 4) only holds under the two assumptions.

We next transform the concurrent-composition representation for the negation of $*$-strong detectability as in Theorem 3.2 to equivalent LTL formulae. One can see that it not very possible to directly do such a transformation, because the condition requires visiting a special types of states q'_2 with $q'_2(L) \neq q'_2(R)$ (see (6)) after visiting a special type of cycles $q'_1 \xrightarrow{s'_2} q'_1$ with $\ell(s'_2) \in \Sigma^+$ (also see (6)). Therefore, we first transform the representation into two conditions, and then transform each of the two conditions to an LTL formula. To this end, we define two transformations, where the first makes an LFSA that is not deadlock-free become deadlock-free, the second swaps the roles of states and events of an LFSA.

Transformation 1. *Consider an LFSA S (1). If S is not deadlock-free, add a fresh sink state $\diamond$, and at each dead state q', add a transition $q' \xrightarrow{\diamond_{q'}} \diamond$ and set $\ell(\diamond_{q'}) = \diamond_{q'}$, and add a self-loop $\diamond \xrightarrow{\diamond} \diamond$ and set $\ell(\diamond) = \diamond$. The modified LFSA is denoted by $S^{\diamond}$. If S is deadlock-free, also denote S by $S^{\diamond}$.*

See Fig. 18 as an illustrative example.

Transformation 2. *Consider an LFSA S (1). Add a fresh event $\diamond$ on the input arrow of each initial state, and then swap the roles of states and events. The modified LFSA is denoted by S^{se}.*

The following fact will be useful.

Fact 1. *One can transform the reachability between two states in an LFSA to an LTL formula.*

Consider an LFSA S (1) and two states q_1, q_2. We apply the following three transformations on S successively: replace the initial state set by $\{q_1\}$, apply Transformation 1, and then apply Transformation 2. Denote the current deadlock-free automaton by $\bar{S}$. One then has q_2 is reachable from q_1 in S if and only if in $\bar{S}$ there is an infinite trace satisfying the LTL formula

$$\bigcirc\Diamond q_2. \tag{23}$$

With slight abuse of notation, we denote this LTL formula as follows: in S, $q_1 \to \bigcirc\Diamond q_2$.

It can be seen that the condition in Theorem 3.2 for the negation of $*$-strong detectability holds if and only if at least one of the following two conditions holds:

(i) In $\mathrm{CC_A}(S)$, there exists a run

$$q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{s'_2} q'_1 \tag{24}$$

satisfying

$$q'_0 \in Q'_0;\ q'_1 \in Q';\ s'_1, s'_2 \in (E')^*; \tag{25a}$$

$$\ell(s'_2) \in \Sigma^+;\ q'_1(L) \neq q'_1(R). \tag{25b}$$

(ii) In $\mathrm{CC_A}(S)$, remove all states q' with $q'(L) \neq q'(R)$ and all their ingoing and outgoing transitions, make all remaining states initial, denote the current modification by $\mathrm{CC}^{\mathrm{diag}}_{\mathrm{A}}(S)$, there is a run

$$q'_0 \xrightarrow{s'_1} q'_1 \xrightarrow{s'_2} q'_1 \tag{26}$$

satisfying q'_0 is reachable in $\mathrm{CC_A}(S)$; $\ell(s'_2) \in \Sigma^+$; and in $\mathrm{CC_A}(S)$, some state q'_2 with $q'_2(L) \neq q'_2(R)$ is reachable from q'_1. Note that q'_0 may not be any initial state of $\mathrm{CC_A}(S)$.

Apply Transformation 1 to $\mathrm{CC_A}(S)$ and then obtain $\mathrm{CC}^{\diamond}_{\mathrm{A}}(S)$. Change each state q' of $\mathrm{CC}^{\diamond}_{\mathrm{A}}(S)$ to a transition $\diamond \xrightarrow{q'} \diamond$ and then obtain $\widehat{\mathrm{CC}^{\diamond}_{\mathrm{A}}}(S)$ (this transformation is called a $\hat{\cdot}$-transformation). Then each run $q'_0 \xrightarrow{e'_1} q'_1 \cdots$ of $\mathrm{CC}^{\diamond}_{\mathrm{A}}(S)$ becomes a run $\diamond \xrightarrow{q'_0} \diamond \xrightarrow{e'_1} \diamond \xrightarrow{q'_1} \cdots$ of $\widehat{\mathrm{CC}^{\diamond}_{\mathrm{A}}}(S)$.

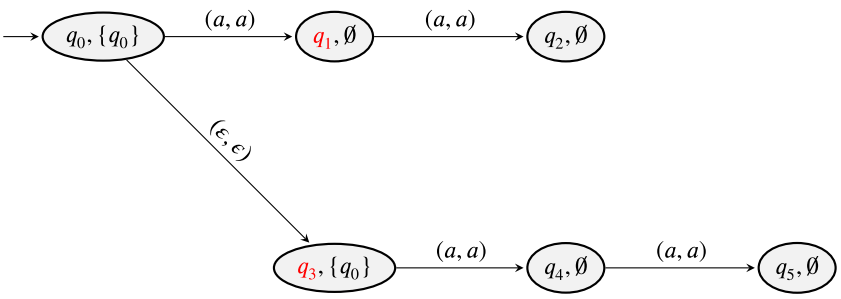


Fig. 17. Part of $CC_A(S_{5\epsilon}, S^{\epsilon}_{5\,\mathrm{dss\,obs}})$ (all reachable states illustrated) corresponding to the LFSA S_5 in Fig. 12.

With these transformations, condition (i) can be equivalently represented by the LTL formula: in $\widehat{CC^{\diamond}_A}(S)$, there is an infinite trace satisfying

$$\bigvee_{e'\in E'_o} \bigvee_{\substack{q'\in Q' \\ q'(L)\neq q'(R)}} (\Box\Diamond e') \wedge (\Box\Diamond q'), \tag{27}$$

where as usual, $\Box\Diamond$ denotes *infinitely often*.

Eq. (27) means that in $CC_A(S)$, there is a reachable observable transition cycle containing at least one state with different components.

On $CC^{\mathrm{diag}}_A(S)$, first apply Transformation 1 and then apply the $\hat{}$-transformation to obtain $\widehat{CC^{\mathrm{diag}\diamond}_A}(S)$. Then condition (ii) can be equivalently represented as the LTL formula: for at least one state $q'_1 \in Q'$ with $q'_1(L) = q'_1(R)$ and at least one state $q'_2 \in Q'$ with $q'_2(L) \neq q'_2(R)$, in $\widehat{CC^{\mathrm{diag}\diamond}_A}(S)$, there is an infinite trace satisfying

$$\bigvee_{e'\in E'_o} (\Box\Diamond e') \wedge (\Box\Diamond q'_1), \tag{28}$$

and

$$\text{in } CC^{\diamond}_A(S),\ (\Diamond q'_1) \wedge (q'_1 \rightarrow \bigcirc\Diamond q'_2). \tag{29}$$

Eq. (28) means that in $CC_A(S)$, there is a reachable observable transition cycle containing q'_1 and each state of the cycle has two components equal. Eq. (29) means that in $CC_A(S)$, q'_1 is reachable (corresponding to $\Diamond q'_1$), q'_2 is reachable from q'_1 (corresponding to $q'_1 \rightarrow \bigcirc\Diamond q'_2$).

Until now, we have represented the condition for the negation of $*$-strong detectability shown in Theorem 3.2 as equivalent LTL formulae. The same can be done for the condition for the negation of ω-strong detectability similarly. One can see that it is much more concise to directly use the concurrent-composition representation shown in Theorem 3.2 to verify strong detectability than using LTL formulae.

Remark 8.1. Transformation 1 preserves $*$-strong detectability of LFSAs. Consider an LFSA S (1) that is not deadlock-free. Apply Transformation 1 on S to obtain $S^{\diamond}$. Then S is not $*$-strongly detectable if and only if $S^{\diamond}$ is not $*$-strongly detectable. To see this, assume S is not $*$-strongly detectable, then by Theorem 3.2, in $CC_A(S)$, there exists a run (6) satisfying (7). This run (6) is also a run of $CC_A(S^{\diamond})$ because all transitions of $CC_A(S)$ are transitions of $CC_A(S^{\diamond})$. Hence $S^{\diamond}$ is not $*$-strongly detectable either. Conversely, assume $S^{\diamond}$ is not $*$-strongly detectable, then by Theorem 3.2, in $CC_A(S^{\diamond})$, there exists a run (6) satisfying (7). Denote the ending state of this run (6) by (q_1, q_2), where $q_1 \neq q_2$. Neither q_1 nor q_2 can be $\diamond$, because if $q_1 = \diamond$, all ingoing transitions must have events $\diamond$ or $\diamond_q$ with q being dead in S, but all ingoing transitions of q_2 must have events as events of S, so cannot have label $\diamond$ or any $\diamond_{q'}$. Therefore, q_1 and q_2 must be states of S. Then this (6) is a run of $CC_A(S)$, so S is not $*$-strongly detectable either.

Remark 8.2. Transformation 1 does not preserve ω-strong detectability, or diagnosability, or predictability of LFSAs. Reconsider LFSA $\bar{S}_2$ (Fig. 4(a)) that is obtained from the LFSA S_2 in Fig. 2(a) by removing the self-loop on state q_1. $\bar{S}_2$ is ω-strongly detectable, but $\bar{S}^{\diamond}_2$ shown in Fig. 18 is not.

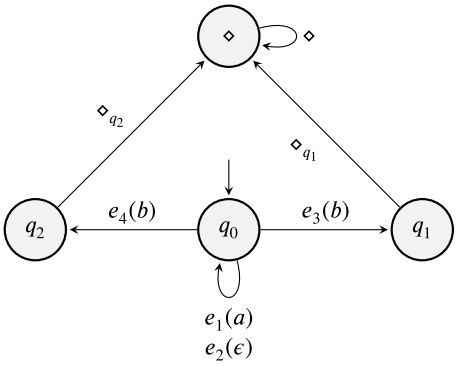


Fig. 18. LFSA $\bar{S}^{\diamond}_2$, obtained by applying Transformation 1 on $\bar{S}_2$ in Fig. 4(a).

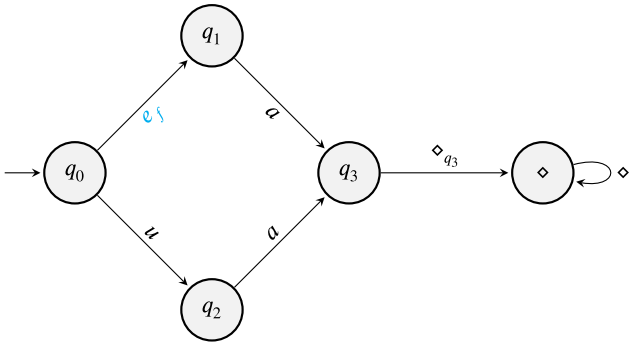


Fig. 19. LFSA $S^{\diamond}_6$ and LFSA S_6 (after removing state $\diamond$ and its ingoing transition from q_3 and self-loop), where e_f is faulty, $\ell(e_f) = \ell(u) = \epsilon$, $\ell(a) = a$.

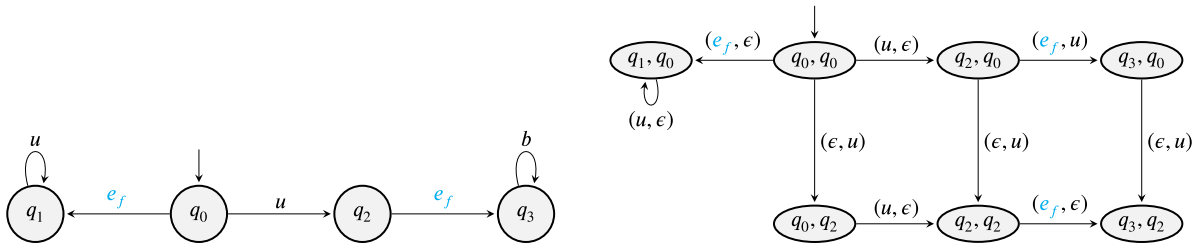
Consider the LFSA S_6 and $S^{\diamond}_6$ shown in Fig. 19, where S_6 does not contain state $\diamond$. S_6 is $\{e_f\}$-diagnosable, but $S^{\diamond}_6$ is not.

Consider $\bar{S}_6$ obtained from S_6 by only keeping states q_0, q_1, q_2 and the transitions between them. $\bar{S}_6$ is $\{e_f\}$-predictable but $\bar{S}^{\diamond}_6$ is not.

8.2. Diagnosability

In Bérard et al. (2018), the negation of a notion called trace-diagnosability was equivalently represented in LTL for LFSAs, where the trace-diagnosability is strictly weaker than the diagnosability as in Definition 10 (called uniform diagnosability in Bérard et al. (2018)) even for LFSAs. The tool used for transforming the trace-diagnosability to an LTL formula is coincidently equivalent to the generalized twin-plant proposed in Cassez and Tripakis (2008).

By using concurrent composition, the necessary and sufficient condition for trace-diagnosability of LFSAs given by Bérard et al. (2018)



(a) LFSA $\mathcal{S}_7$. (b) Part of concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{7\mathrm{f}}, \mathcal{S}_{7\mathrm{n}})$.

Fig. 20. LFSA $\mathcal{S}_7$ given in Bérard et al. (2018) (left), where e_f is faulty, $\ell(e_f) = \ell(u) = \epsilon$, $\ell(b) = b$, and part of concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{7\mathrm{f}}, \mathcal{S}_{7\mathrm{n}})$ (right).

can be restated as follows: an LFSA $\mathcal{S}$ (1) is not trace-diagnosable if and only if in $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\mathrm{f}}, \mathcal{S}_{\mathrm{n}})$, there exists a run (10) satisfying

$$q_0' \in Q_0';\ e'(L) \in E_f;\ |s_3'(L)| > 0;\ |s_3'(R)| > 0. \tag{30}$$

Comparing this condition with Theorem 4.2, one can see that (30) is slightly but strictly stronger than (11), hence one can expect that trace-diagnosability is strictly weaker than diagnosability.

Consider the following LFSA $\mathcal{S}_7$ given in Bérard et al. (2018) (shown in Fig. 20(a)). $\mathcal{S}_7$ is not $\{e_f\}$-diagnosable because for all $n \in \mathbb{N}$, choose run $q_0 \xrightarrow{e_f} q_1 \left(\xrightarrow{u} q_1\right)^n$ and run $q_0 \xrightarrow{u} q_2$, the latter contains no faulty event but they produce the same label sequence ϵ. $\mathcal{S}_7$ is trace-diagnosable, because there are totally two faulty infinite runs $q_0 \xrightarrow{e_f} q_1 \left(\xrightarrow{u} q_1\right)^{\omega}$ and $q_0 \xrightarrow{u} q_2 \xrightarrow{e_f} q_3 \left(\xrightarrow{b} q_3\right)^{\omega}$, they produce different label sequences ϵ and b^{ω}. By the concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{7\mathrm{f}}, \mathcal{S}_{7\mathrm{n}})$ shown in Fig. 20(b), one can obtain the same conclusion: there is a run $(q_0, q_0) \xrightarrow{(e_f,\epsilon)} (q_1, q_0) \xrightarrow{(u,\epsilon)} (q_1, q_0)$ as in (10) satisfying (11), but there is no run as in (10) satisfying (30).

Next, by using the concurrent-composition representation for the negation of diagnosability shown in Theorem 4.2, we equivalently represent the negation of diagnosability as an LTL formula.

Consider an LFSA (1) and the corresponding concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\mathrm{f}}, \mathcal{S}_{\mathrm{n}})$. Apply Transformation 1 on $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\mathrm{f}}, \mathcal{S}_{\mathrm{n}})$ to obtain $\mathrm{CC}_{\mathrm{A}}^{\diamond}(\mathcal{S}_{\mathrm{f}}, \mathcal{S}_{\mathrm{n}})$ that is deadlock-free. Then we can use the LTL model checking technique to verify diagnosability. By Theorem 4.2, $\mathcal{S}$ is not E_f-diagnosable if and only if in $\mathrm{CC}_{\mathrm{A}}^{\diamond}(\mathcal{S}_{\mathrm{f}}, \mathcal{S}_{\mathrm{n}})$, there is an infinite trace satisfying the following LTL formula

$$\left(\bigvee_{e_f \in E_f} \Diamond(e_f, \epsilon)\right) \wedge \left(\bigvee_{(e_1,e_2)\in\tilde{E}'} \Box\Diamond(e_1, e_2)\right), \tag{31}$$

where $\tilde{E}' = \{(\tilde{e}_1, \tilde{e}_2) | \tilde{e}_1 \in E_o, \tilde{e}_2 \in E_o \setminus E_f, \ell(\tilde{e}_1) = \ell(\tilde{e}_2)\} \cup \{(\tilde{e}_1, \epsilon) | \tilde{e}_1 \in E_{uo}\}$.

In Bérard et al. (2018, Section 2.2.1), it was shown that the Büchi automaton for the LTL formula (eqn. (φ_{diag}) on Page 325) corresponding to the negation of trace-diagnosability satisfies that its number of states does not depend on the number of events of $\mathcal{S}$, so an NL upper bound on verification of trace diagnosability was obtained. The same holds for diagnosability and formula (31) because (31) is quite similar to (φ_{diag}). However, directly by using the concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\mathrm{f}}, \mathcal{S}_{\mathrm{n}})$, an NL upper bound on verification of diagnosability can also be obtained (Zhang, 2021). On the other hand, the fact that the number of states of the Büchi automaton corresponding to (φ_{diag}) does not depend on the number of events of $\mathcal{S}$ was seen by the authors of paper (Bérard et al., 2018) but cannot be seen by model checking toolboxes, and it is generally PSPACE-complete for LTL model checking in the number of events, even for a subclass of LTL formulae that only contain temporal operator $\Diamond$ and propositional operator $\wedge$, the LTL model checking problem is already NP-hard (Bauland et al., 2011; Sistla & Clarke, 1985), so one usually cannot expect efficiency improvement by using LTL model checking toolboxes compared with directly using concurrent composition. Hence in order to verify diagnosability, LTL model checking techniques do not bring in efficiency improvement.

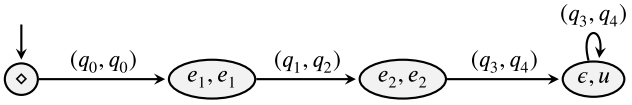


Fig. 21. Part of $\mathrm{CC}_{\mathrm{A}}^{\mathrm{se}}(\mathcal{S}_{3\mathrm{f}}, \mathcal{S}_{3\mathrm{n}})$, where $\mathcal{S}_3$ is shown in Fig. 8(a).

In Zhao et al. (2022), a modified notion of diagnosability was studied (see Zhao et al., 2022, Definition 3), where the diagnosability was obtained from Definition 10 by changing $|s'| > k$ to $|\ell^{\diamond}(s')| > k$. Also in Zhao et al. (2022, Theorem 1), this modified notion was equivalently represented by a hyper LTL formula which is exactly the notion of trace-diagnosability studied in Bérard et al. (2018). Hence, the trace-diagnosability was represented by a hyper LTL formula in Zhao et al. (2022).

8.3. Predictability

Now we represent the negation of predictability of an LFSAs $\mathcal{S}$ (1) equivalently as an LTL formula based on the concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\mathrm{n}}, \mathcal{S}_{\mathrm{n}})$ by using Theorem 5.1, in which the negation of predictability was presented as the reachability of a special type of states q_1' (defined in (14)) in $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\mathrm{n}}, \mathcal{S}_{\mathrm{n}})$. We apply Transformation 2 on $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\mathrm{n}}, \mathcal{S}_{\mathrm{n}})$ to obtain $\mathrm{CC}_{\mathrm{A}}^{\mathrm{se}}(\mathcal{S}_{\mathrm{n}}, \mathcal{S}_{\mathrm{n}})$, and then apply Transformation 1 on $\mathrm{CC}_{\mathrm{A}}^{\mathrm{se}}(\mathcal{S}_{\mathrm{n}}, \mathcal{S}_{\mathrm{n}})$ to obtain $\mathrm{CC}_{\mathrm{A}}^{\mathrm{se}\diamond}(\mathcal{S}_{\mathrm{n}}, \mathcal{S}_{\mathrm{n}})$. By Theorem 5.1, $\mathcal{S}$ is not E_f-predictable if and only if in $\mathrm{CC}_{\mathrm{A}}^{\mathrm{se}\diamond}(\mathcal{S}_{\mathrm{n}}, \mathcal{S}_{\mathrm{n}})$, there exists an infinite trace satisfying the following LTL formula

$$\bigvee_{\substack{q_1'(L)\in\partial S_{\mathrm{n}} \\ q_1'(R)\in\nabla S_{\mathrm{n}}}} \Diamond(q_1'(L), q_1'(R)), \tag{32}$$

where $\partial S_{\mathrm{n}} = \{q \in Q | (q, e_f, q') \in \delta$ for some $e_f \in E_f$ and $q' \in Q\}$, $\nabla S_{\mathrm{n}} = \{q \in Q |$ there is a transition cycle reachable from q in $S_{\mathrm{n}}\}$. The LTL checking problem for (32) (containing only temporal operator $\Diamond$ and propositional operator $\vee$) belongs to NL (Bauland et al., 2011).

Example 12. Reconsider the LFSA $\mathcal{S}_3$ in Fig. 8(a). Part of the concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{3\mathrm{n}}, \mathcal{S}_{3\mathrm{n}})$ can be obtained from Fig. 8(b) by removing the transition with event (f, ϵ). Then the corresponding $\mathrm{CC}_{\mathrm{A}}^{\mathrm{se}}(\mathcal{S}_{3\mathrm{n}}, \mathcal{S}_{3\mathrm{n}})$ (the same as $\mathrm{CC}_{\mathrm{A}}^{\mathrm{se}\diamond}(\mathcal{S}_{3\mathrm{n}}, \mathcal{S}_{3\mathrm{n}})$) is shown in Fig. 21, where (q_3, q_4) is reachable, $q_3 \in \partial S_{3\mathrm{n}}$, $q_4 \in \nabla S_{3\mathrm{n}}$. Then one has $\mathcal{S}_3$ is not $\{f\}$-predictable.

8.4. Standard state-based opacity and strong state-based opacity

Consider an LFSA $\mathcal{S}$ (1). By Theorem 6.1, checking CSO of $\mathcal{S}$ is to check reachability of a special type of states in the observer $\mathcal{S}_{\mathrm{obs}}$. By Theorems. 6.2, 6.4, and 6.5, checking ISO, InfSO, and KSO of $\mathcal{S}$ are to check reachability of special types of states in concurrent composition $\mathrm{CC}_{\mathrm{A}}(\mathcal{S}_{\epsilon}, \mathcal{S}_{\mathrm{obs}}^{\epsilon})$. Similarly, by Theorems 7.2, 7.3, 7.4, and 7.5, checking

SCSO, SISO, SInfSO, and SKSO of S are to check reachability of special types of states in concurrent composition $\mathrm{CC_A}(S_\varepsilon, S^{\varepsilon}_{\mathrm{dss\,obs}})$. Hence similar to predictability, one can transform the four standard versions of state-based opacity and four strong versions of state-based opacity as LTL formulae based on the corresponding observers and concurrent compositions. We omit similar procedure.

9. Conclusion

In this paper, a unified concurrent-composition method was given to verify plenty of inference-based properties and concealment-based properties in labeled finite-state automata (LFSAs). Compared with the previous verification algorithms in the literature, the concurrent-composition method does not depend on assumptions and is more efficient. These results showed that many inference-based properties and concealment-based properties can be unified into one mathematical framework, although the two categories of properties look quite different. This kind of unification between the two categories has never been revealed before. It is interesting to explore other applications of the concurrent-composition method, e.g., what other properties could be verified by the method, what other kinds of models in discrete-event systems could be dealt with by the method, and what other problems (e.g., enforcement) could be solved by the method. As an example, the resilience of an LFSA S was verified using the concurrent composition S and its observer S_{obs} (see the product automaton in the proof of Fabre, 2022, Theorem 4).

Recently, along with the line of using the concurrent-composition method to verify the negation of strong versions of detectability, a new theory framework for enforcing properties was proposed (Zhang, 2023a). The basic philosophy of the new framework is choosing a subset of controllable events/transitions to disable in order to cut off all runs in the corresponding concurrent compositions that violate a property of interest. Compared with the classical supervisory control framework, the new theory framework has better scalability and wider implementability. The new framework can be implemented in polynomial time on properties verifiable in polynomial time, while the supervisory control framework cannot be implemented in polynomial time because its implementation is EXPTIME-complete (e.g., see Haar et al., 2017 on diagnosability of LFSAs and Haar et al., 2020 on predictability of LFSAs) and P $\subsetneq$ EXPTIME. The new framework can be implemented on decidable properties in partially-observed systems with finitely many events, but currently supervisory control can be fully implemented only on LFSAs, because for more general systems, the observers (the basics of supervisors) therein are generally not computable, e.g., labeled timed automata (Cassez, 2009; Zhang, 2022) and labeled Petri nets (Bryans et al., 2008). The new property enforcement framework for discrete-event systems has just been established.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article

Appendix

We briefly review the twin-plant method proposed by Jiang et al. (2001) and the verifier method proposed by Yoo and Lafortune (2002) used for verifying diagnosability and show that they usually do not work without the two assumptions of liveness/deadlock-freeness and divergence-freeness. We also briefly show the coincident similarity between the concurrent composition and the generalized version of the twin-plant proposed by Cassez and Tripakis (2008).

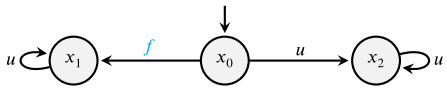


Fig. 22. LFSA S_8, where $\ell(f) = \ell(u) = \epsilon$.

For brevity, we consider an LFSA $S = (Q, E, \delta, Q_0, \Sigma, \ell)$ in which $\ell|_{E_o}$ is the identity mapping and Q_0 is a singleton and denoted by $\{q_0\}$. Recall that E_o is the set of observable events, and $E_{uo} = E \setminus E_o$ is the set of unobservable events. Consider a single faulty event $f \in E_{uo}$.

The twin-plant TwPl_S of S proposed by Jiang et al. (2001) is constructed as follows:

(1) Construct the automaton $S_\phi = (Q_\phi, E_o, (x_0, \phi), \delta_\phi)$, where the initial state is $(x_0, \phi) \in Q_\phi$, $Q_\phi = Q \times \{\phi, F\}$; for all $(x_1, \phi), (x_2, l_2) \in Q_\phi$ and $t \in E_o$, $((x_1, \phi), t, (x_2, l_2)) \in \delta_\phi$ if and only if there is a run $x_1 \xrightarrow{st} x_2$ in S such that $s \in (E_{uo})^*$, and $l_2 = F$ if and only if f appears in at least one such s; for all $(x_1, F), (x_2, l_2) \in Q_\phi$ and $t \in E_o$, $((x_1, F), t, (x_2, l_2)) \in \delta_\phi$ if and only if there is a run $x_1 \xrightarrow{st} x_2$ in S such that $s \in E^*_{uo}$ and $l_2 = F$.[9]

(2) The twin-plant TwPl_S is the parallel composition $S_\phi \| S_\phi$ of S_ϕ with itself, where the parallel composition is as in Cassandras and Lafortune (2008, Page 80). In this special case, $S_\phi \| S_\phi$ is almost the same as the self-composition $\mathrm{CC_A}(S_\phi)$ because S_ϕ contains no unobservable events. After replacing each event e_o in $S_\phi \| S_\phi$ by (e_o, e_o), $\mathrm{CC_A}(S_\phi)$ is obtained.

Proposition 9.1 (*Jiang et al., 2001*)**.** *A live and divergence-free S is $\{f\}$-diagnosable if and only if in TwPl_S all states of all cycles are of the form $((-, l_1), (-, l_2))$ with $l_1 = l_2$.*

Example 13. Proposition 9.1 does not generally hold for S that is not live or divergence-free. Consider the LFSA S_8 in Fig. 22. By definition, $S_{8\phi}$ consists of only the initial state (x_0, ϕ). Hence by Proposition 9.1 S_8 is $\{f\}$-diagnosable vacuously. However, by definition, S_8 is not $\{f\}$-diagnosable.

The verifier $\mathrm{Ver}_S = (Q_{\mathrm{Ver}}, E, (x_0, N, x_0, N), \delta_{\mathrm{Ver}})$ of S proposed by Yoo and Lafortune (2002) is constructed as follows: $Q_{\mathrm{Ver}} = Q \times \{N, F\} \times Q \times \{N, F\}$, for all $(x_1, l_1, x_2, l_2) \in Q_{\mathrm{Ver}}$, $\sigma_o \in E_o$, and $\sigma_{uo} \in E_{uo} \setminus \{f\}$,

(i) $((x_1, l_1, x_2, l_2), f, (x'_1, F, x_2, l_2)) \in \delta_{\mathrm{Ver}}$ if and only if $(x_1, f, x'_1) \in \delta$,
(ii) $((x_1, l_1, x_2, l_2), f, (x_1, l_1, x'_2, F)) \in \delta_{\mathrm{Ver}}$ if and only if $(x_2, f, x'_2) \in \delta$,
(iii) $((x_1, l_1, x_2, l_2), f, (x'_1, F, x'_2, F)) \in \delta_{\mathrm{Ver}}$ if and only if $(x_1, f, x'_1), (x_2, f, x'_2) \in \delta$,
(iv) $((x_1, l_1, x_2, l_2), \sigma_{uo}, (x'_1, l_1, x_2, l_2)) \in \delta_{\mathrm{Ver}}$ if and only if $(x_1, \sigma_{uo}, x'_1) \in \delta$,
(v) $((x_1, l_1, x_2, l_2), \sigma_{uo}, (x_1, l_1, x'_2, l_2)) \in \delta_{\mathrm{Ver}}$ if and only if $(x_2, \sigma_{uo}, x'_2) \in \delta$,
(vi) $((x_1, l_1, x_2, l_2), \sigma_{uo}, (x'_1, l_1, x'_2, l_2)) \in \delta_{\mathrm{Ver}}$ if and only if $(x_1, \sigma_{uo}, x'_1), (x_2, \sigma_{uo}, x'_2) \in \delta$,
(vii) $((x_1, l_1, x_2, l_2), \sigma_o, (x'_1, l_1, x'_2, l_2)) \in \delta_{\mathrm{Ver}}$ if and only if $(x_1, \sigma_o, x'_1), (x_2, \sigma_o, x'_2) \in \delta$.

[9] Here F denotes propagation of f, i.e., along every run of S_ϕ, once a state has its right component equal to F, then all subsequent states have their right components equal to F.

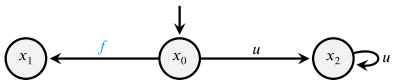


Fig. 23. LFSA $\mathcal{S}_9$, where $\ell(f) = \ell(u) = \epsilon$.

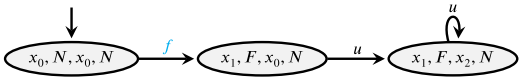


Fig. 24. Part of $\text{Ver}_{\mathcal{S}_9}$ corresponding to the LFSA $\mathcal{S}_9$ in Fig. 23.

Proposition 9.2 (*Yoo & Lafortune, 2002*). *A live and divergence-free $\mathcal{S}$ is $\{f\}$-diagnosable if and only if in $\text{Ver}_{\mathcal{S}}$ all states of all cycles are of the form $(-, l_1, -, l_2)$ with $l_1 = l_2$.*[10]

Example 14. Proposition 9.2 does not generally hold for $\mathcal{S}$ that is not live or divergence-free. Consider the LFSA $\mathcal{S}_9$ in Fig. 23. Part of $\text{Ver}_{\mathcal{S}_9}$ is shown in Fig. 24. The cycle $(x_1, F, x_2, N) \xrightarrow{u} (x_1, F, x_2, N)$ in $\text{Ver}_{\mathcal{S}_9}$ contradicts the condition in Proposition 9.2, hence by Proposition 9.2 $\mathcal{S}_9$ is not $\{f\}$-diagnosable. However, by definition, $\mathcal{S}_9$ is $\{f\}$-diagnosable vacuously.

The generalized twin-plant

$$\overline{\text{TwPl}}_{\mathcal{S}} = (Q_{\text{Ver}}, \{(\sigma_o, \sigma_o) | \sigma_o \in E_o\} \cup (T_{uo} \times \{\epsilon\}) \cup (\{\epsilon\} \times (E_{uo} \setminus \{f\})), (x_0, N, x_0, N), \delta_{\overline{\text{TwPl}}})$$

of $\mathcal{S}$ proposed by Cassez and Tripakis (2008) is constructed as follows: for all $(x_1, l_1, x_2, l_2) \in Q_{\text{Ver}}$, (σ_o, σ_o) with $\sigma_o \in E_o$, and $\sigma_{uo} \in E_{uo} \setminus \{f\}$,

(a) $((x_1, l_1, x_2, l_2), (f, \epsilon), (x_1', F, x_2, l_2)) \in \delta_{\text{Ver}}$ if and only if $(x_1, f, x_1') \in \delta$,
(b) $((x_1, l_1, x_2, l_2), (\sigma_{uo}, \epsilon), (x_1', l_1, x_2, l_2)) \in \delta_{\text{Ver}}$ if and only if $(x_1, \sigma_{uo}, x_1') \in \delta$,
(c) $((x_1, l_1, x_2, l_2), (\epsilon, \sigma_{uo}), (x_1, l_1, x_2', l_2)) \in \delta_{\text{Ver}}$ if and only if $(x_2, \sigma_{uo}, x_2') \in \delta$,
(d) $((x_1, l_1, x_2, l_2), (\sigma_o, \sigma_o), (x_1', l_1, x_2', l_2)) \in \delta_{\text{Ver}}$ if and only if $(x_1, \sigma_o, x_1'), (x_2, \sigma_o, x_2') \in \delta$.

There is no state of the form $(-, -, -, F)$ reachable in $\overline{\text{TwPl}}_{\mathcal{S}}$.

Proposition 9.3 (*Cassez & Tripakis, 2008*). *An LFSA $\mathcal{S}$ (1) is not $\{f\}$-diagnosable if and only if in $\overline{\text{TwPl}}_{\mathcal{S}}$ there is a reachable cycle in which all states are of the form $(-, F, -, N)$ and there is at least one event of the form $(\sigma, -)$ with $\sigma \in E$.*

Example 15. Consider $\mathcal{S}_9$ in Fig. 23. $\overline{\text{TwPl}}_{\mathcal{S}_9}$ is shown in Fig. 25. In Fig. 25, there is no reachable cycle as in Proposition 9.3. In the cycle $(x_1, F, x_2, N) \xrightarrow{(\epsilon, u)} (x_1, F, x_2, N)$, in the unique event (ϵ, u), the left component is ϵ. Hence by Proposition 9.3 $\mathcal{S}_9$ is $\{f\}$-diagnosable.

It is easy to see that the concurrent composition $\text{CC}_{\text{A}}(\mathcal{S}_{\text{f}}, \mathcal{S}_{\text{n}})$ in the current paper used for verifying diagnosability is coincidently similar to the generalized version of twin-plant $\overline{\text{TwPl}}_{\mathcal{S}}$ proposed by Cassez and Tripakis (2008). After removing all F's and N's from $\overline{\text{TwPl}}_{\mathcal{S}}$, $\text{CC}_{\text{A}}(\mathcal{S}_{\text{f}}, \mathcal{S}_{\text{n}})$ is obtained.

[10] In $\text{Ver}_{\mathcal{S}}$, if there is a cycle containing a state of the form $(-, l_1, -, l_2)$ with $l_1 \neq l_2$, then either (1) all states in the cycle are of the form $(-, F, -, N)$ or (2) all states in the cycle are of the form $(-, N, -, F)$.

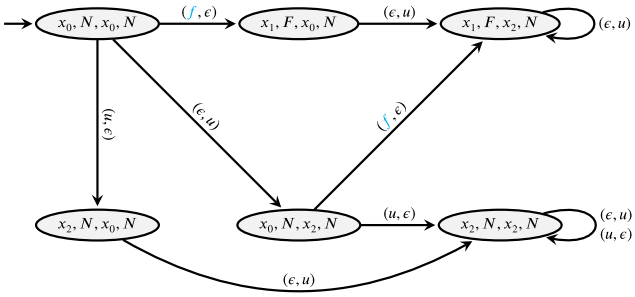


Fig. 25. Part of $\overline{\text{TwPl}}_{\mathcal{S}_9}$ corresponding to the LFSA $\mathcal{S}_9$ in Fig. 23.

References

Angulo, M. T., Aparicio, A., & Moog, C. H. (2019). Structural accessibility and structural observability of nonlinear networked systems. *IEEE Transactions on Network Science and Engineering*, online.

Baier, C., & Katoen, J. P. (2008). *Principles of model checking*. The MIT Press.

Balun, J., & Masopust, T. (2021). Comparing the notions of opacity for discrete-event systems. *Discrete Event Dynamic Systems*, *31*(4), 553–582.

Balun, J., & Masopust, T. (2022a). On transformations among opacity notions. In *2022 IEEE international conference on systems, man, and cybernetics* (pp. 3012–3017a).

Balun, J., & Masopust, T. (2022b). On verification of weak and strong K-step opacity for discrete-svent systems. In *2022 16th international workshop on discrete event systems*.

Basilio, J. C., Hadjicostis, C. N., & Su, R. (2021). Analysis and control for resilience of discrete event systems: Fault diagnosis, opacity and cyber security. *Foundations and Trends® in Systems and Control*, *8*(4), 285–443.

Bauland, M., Mundhenk, M., Schneider, T., Schnoor, H., Schnoor, I., & Vollmer, H. (2011). The tractability of model checking for LTL: The good, the bad, and the ugly fragments. *ACM Transactions on Computational Logic*, *12*(2), 1–28.

Bérard, B., Haar, S., Schmitz, S., & Schwoon, S. (2018). The complexity of diagnosability and opacity verification for Petri nets. *Fundamenta Informaticae*, *161*(4), 317–349.

Broy, M., Jonsson, B., Katoen, J. P., Martin, L., & Pretschner, A. (2005). *Lecture notes in computer science, Model-based testing of reactive systems: Advanced lectures*. Secaucus, NJ, USA: Springer-Verlag New York, Inc.

Bryans, J. W., Koutny, M., Mazaré, L., & Ryan, P. Y. A. (2008). Opacity generalised to transition systems. *International Journal of Information Security*, *7*(6), 421–435.

Bryans, J. W., Koutny, M., & Ryan, P. Y. A. (2005). Modelling opacity using Petri nets. In *Electronic notes in theoretical computer science: vol. 121, Proceedings of the 2nd international workshop on security issues with Petri nets and other computational models* (pp. 101–115).

Cabasino, M. P., Giua, A., Lafortune, S., & Seatzu, C. (2012). A new approach for diagnosability analysis of Petri nets using verifier nets. *IEEE Transactions on Automatic Control*, *57*(12), 3104–3117.

Caines, P. E., Greiner, R., & Wang, S. (1988). Dynamical logic observers for finite automata. In *Proceedings of the 27th IEEE conference on decision and control, vol. 1* (pp. 226–233).

Caines, P. E., Greiner, R., & Wang, S. (1991). Classical and logic-based dynamic observers for finite automata. *IMA Journal of Mathematical Control and Information*, *8*(1), 45–80.

Cassandras, C. G., & Lafortune, S. (2008). *Introduction to discrete event systems* (2nd ed.). NY: Springer New York.

Cassez, F. (2009). The dark side of timed opacity. In Jong Hyuk Park, Hsiao-Hwa Chen, Mohammed Atiquzzaman, Changhoon Lee, Tai hoon Kim, & Sang-Soo Yeo (Eds.), *Advances in information security and assurance* (pp. 21–30). Berlin, Heidelberg: Springer Berlin Heidelberg.

Cassez, F., Dubreil, J., & Marchand, H. (2009). Dynamic observers for the synthesis of opaque systems. In *Automated technology for verification and analysis* (pp. 352–367). Berlin, Heidelberg: Springer Berlin Heidelberg.

Cassez, F., & Tripakis, S. (2008). Fault diagnosis with static and dynamic observers. *Fundamenta Informaticae*, *88*(4), 497–540.

Chaum, D. (1988). The dining cryptographers problem: Unconditional sender and recipient untraceability. *Journal of Cryptology*, *1*(1), 65–75.

Conte, G., Moog, C. H., & Perdon, A. M. (2007). *Algebraic methods for nonlinear control systems* (2nd ed.). Springer-Verlag London.

Fabre, E. (2022). Resilience in discrete event systems. In *16th IFAC workshop on discrete event systems, vol. 55* (pp. 250–255).

Falcone, Y., & Marchand, H. (2015). Enforcement and validation (at runtime) of various notions of opacity. *Discrete Event Dynamic Systems: Theory & Applications*, *25*, 531–570.

Genc, S., & Lafortune, S. (2009). Predictability of event occurrences in partially-observed discrete-event systems. *Automatica*, *45*(2), 301–311.

Gill, A. (1962). *Introduction to theory of finite-state machines*. Europe, New York, United States: McGraw–Hill Education.

Góes, R. M., Rawlings, B. C., Recker, N., Willett, G., & Lafortune, S. (2018). Demonstration of indoor location privacy enforcement using obfuscation. In *14th IFAC workshop on discrete event systems, vol. 51* (pp. 145–151).

Haar, S., Haddad, S., Melliti, T., & Schwoon, S. (2017). Optimal constructions for active diagnosis. *Journal of Computer and System Sciences*, *83*(1), 101–120.

Haar, S., Haddad, S., Schwoon, S., & Ye, L. (2020). Active prediction for discrete event systems. In Nitin Saxena, & Sunil Simon (Eds.), *Leibniz international proceedings in informatics (LIPIcs)*: *vol. 182*, *40th IARCS annual conference on foundations of software technology and theoretical computer science* (pp. 48:1–48:16). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

Han, X., Zhang, K., Zhang, J., Li, Z., & Chen, Z. (2023). Strong current-state and initial-state opacity of discrete-event systems. *Automatica*, *148*, 110756 (1–8).

Isidori. Nonlinear Control Systems, A. (1995). *Communications and control engineering*. Springer-Verlag London.

Jiang, S., Huang, Z., Chandra, V., & Kumar, R. (2001). A polynomial algorithm for testing diagnosability of discrete-event systems. *IEEE Transactions on Automatic Control*, *46*(8), 1318–1321.

Kalman, R. E. (1960). On the general theory of control systems. In *1st international IFAC congress on automatic and remote control, vol. 1* (pp. 491–502).

Kalman, R. E. (1963). Mathematical description of linear dynamical systems. *Journal of the Society for Industrial and Applied Mathematics Series A Control*, *1*(12), 152–192.

Kibangou, A. Y., Garin, F., & Gracy, S. (2016). Input and state observability of network systems with a single unknown input, 49. (pp. 37–42).

Lafortune, S., Lin, F., & Hadjicostis, C. N. (2018). On the history of diagnosability and opacity in discrete event systems. *Annual Reviews in Control*, *45*, 257–266.

Lin, F. (1994). Diagnosability of discrete event systems and its applications. *Discrete Event Dynamic Systems*, *4*(2), 197–212.

Lin, F. (2011). Opacity of discrete event systems and its applications. *Automatica*, *47*(3), 496–503.

Liu, Y., Liu, Z., Yin, X., & Li, S. (2020). An improved approach for verifying delayed detectability of discrete-event systems. *Automatica*, Article 109291.

Liu, Z., Yin, X., & Li, S. (2019). Improved approaches for verifying I-detectability of discrete-event systems. In *2019 12th Asian control conference* (pp. 248–253).

Ma, Z., Yin, X., & Li, Z. (2021). Verification and enforcement of strong infinite- and K-step opacity using state recognizers. *Automatica*, *133*, Article 109838.

Masopust, T. (2018). Complexity of deciding detectability in discrete event systems. *Automatica*, *93*, 257–261.

Mazaré, L. (2004). Using unification for opacity properties. In *Proceedings of the workshop on issues in the theory of security* (pp. 165–176).

Moore, E. F. (1956). Gedanken-experiments on sequential machines. *Automata Studies, Annals of Mathematics Studies*, *34*, 129–153.

Özveren, C. M., & Willsky, A. S. (1990). Observability of discrete event dynamic systems. *IEEE Transactions on Automatic Control*, *35*(7), 797–806.

Pnueli, A. (1977). The temporal logic of programs. In *18th annual symposium on foundations of computer science* (pp. 46–57).

Qiu, W., & Kumar, R. (2006). Decentralized failure diagnosis of discrete event systems. *IEEE Transactions on Systems, Man, and Cybernetics*, *36*, 384–395.

Rabin, M. O., & Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development*, *3*(2), 114–125.

Ramadge, P. J. (1986). Observability of discrete event systems. In *1986 25th IEEE conference on decision and control* (pp. 1108–1112).

Saboori, A. (2010). *Verification and enforcement of state-based notions of opacity in discrete event systems* (Ph.D. thesis), University of Illinois at Urbana-Champaign.

Saboori, A., & Hadjicostis, C. N. (2007). Notions of security and opacity in discrete event systems. In *2007 46th IEEE conference on decision and control* (pp. 5056–5061).

Saboori, A., & Hadjicostis, C. N. (2009). Verification of K-step opacity and analysis of its complexity. In *Proceedings of the 48h IEEE conference on decision and control (CDC) held jointly with 2009 28th Chinese control conference* (pp. 205–210).

Saboori, A., & Hadjicostis, C. N. (2012). Verification of infinite-step opacity and complexity considerations. *IEEE Transactions on Automatic Control*, *57*(5), 1265–1269.

Saboori, A., & Hadjicostis, C. N. (2013). Verification of initial-state opacity in security applications of discrete event systems. *Information Sciences*, *246*, 115–132.

Sampath, M., Sengupta, R., Lafortune, S., Sinnamohideen, K., & Teneketzis, D. (1995). Diagnosability of discrete-event systems. *IEEE Transactions on Automatic Control*, *40*(9), 1555–1575.

Sasi, Y., & Lin, F. (2018). Detectability of networked discrete event systems. *Discrete Event Dynamic Systems*, *28*(3), 449–470.

Shu, S., & Lin, F. (2011). Generalized detectability for discrete event systems. *Systems & Control Letters*, *60*(5), 310–317.

Shu, S., & Lin, F. (2013a). Delayed detectability of discrete event systems. *IEEE Transactions on Automatic Control*, *58*(4), 862–875a.

Shu, S., & Lin, F. (2013b). I-detectability of discrete-event systems. *IEEE Transactions on Automation Science and Engineering*, *10*(1), 187–196b.

Shu, S., Lin, F., & Ying, H. (2007). Detectability of discrete event systems. *IEEE Transactions on Automatic Control*, *52*(12), 2356–2359.

Silva, M. (2018). On the history of discrete event systems. *Annual Reviews in Control*, *45*, 213–222.

Sipser, M. (1996). *Introduction to the theory of computation* (1st ed.). International Thomson Publishing.

Sistla, A. P., & Clarke, E. M. (1985). The complexity of propositional linear temporal logics. *Journal of the ACM*, *32*(3), 733–749.

Sontag, E. D. (1979). On the observability of polynomial systems, I: Finite-time problems. *SIAM Journal on Control and Optimization*, *17*, 139–151.

Tanwani, A., Shim, H., & Liberzon, D. (2013). Observability for switched linear systems: characterization and observer design. *IEEE Transactions on Automatic Control*, *58*(4), 891–904.

Tong, Y., Li, Z., Seatzu, C., & Giua, A. (2018). Current-state opacity enforcement in discrete event systems under incomparable observations. *Discrete Event Dynamic Systems*, *28*(2), 161–182.

Wintenberg, A., Blischke, M., Lafortune, S., & Ozay, N. (2022). A general language-based framework for specifying and verifying notions of opacity. *Discrete Event Dynamic Systems*, *32*(2), 253–289.

Wonham, W. M. (1985). *Linear multivariable control: A geometric approach* (3rd ed.). Springer-Verlag New York.

Wonham, W. M., & Cai, K. (2019). *Supervisory control of discrete-event systems*. Springer International Publishing.

Wu, Y. (2014). *Verification and enforcement of opacity security properties in discrete event systems* (Ph.D. thesis), University of Michigan.

Wu, Y., & Lafortune, S. (2013). Comparative analysis of related notions of opacity in centralized and coordinated architectures. *Discrete Event Dynamic Systems*, *23*(3), 307–339.

Wu, Y., Raman, V., Lafortune, S., & Seshia, S. A. (2016). Obfuscator synthesis for privacy and utility. In Sanjai Rayadurgam, & Oksana Tkachuk (Eds.), *NASA formal methods* (pp. 133–149). Cham: Springer International Publishing.

Wu, Y., Raman, V., Rawlings, B. C., Lafortune, S., & Seshia, S. A. (2018). Synthesis of obfuscation policies to ensure privacy and utility. *Journal of Automated Reasoning*, *60*(1), 107–131.

Yin, X., & Lafortune, S. (2017). A new approach for the verification of infinite-step and K-step opacity using two-way observers. *Automatica*, *80*, 162–171.

Yoo, T.-S., & Lafortune, S. (2002). Polynomial-time verification of diagnosability of partially observed discrete-event systems. *IEEE Transactions on Automatic Control*, *47*(9), 1491–1495.

Zhang, K. (2017). The problem of determining the weak (periodic) detectability of discrete event systems is PSPACE-complete. *Automatica*, *81*, 217–220.

Zhang, K. (2021). A unified method to decentralized state detection and fault diagnosis/prediction of discrete-event systems. *Fundamenta Informaticae*, *181*, 339–371.

Zhang, K. (2022). Detectability of labeled weighted automata over monoids. *Discrete Event Dynamic Systems*, *32*(3), 435–494.

Zhang, K. (2023a). Polynomial-time verification and enforcement of delayed strong detectability for discrete-event systems. *IEEE Transactions on Automatic Control*, *68*(1), 510–515a.

Zhang, K. (2023b). Removing two fundamental assumptions in verifying strong periodic (D-)detectability of discrete-dvent systems. *IEEE Control Systems Letters*, *7*, 1518–1523b.

Zhang, K., & Feng, L. (2020). Revisiting strong detectability of networked discrete-event systems. In *15th IFAC workshop on discrete event systems, vol. 53* (pp. 21–27).

Zhang, K., & Giua, A. (2019). K-Delayed strong detectability of discrete-event systems. In *Proceedings of the 58th IEEE conference on decision and control* (pp. 7647–7652).

Zhang, K., & Giua, A. (2020). On detectability of labeled Petri nets and finite automata. *Discrete Event Dynamic Systems*, *30*(3), 465–497.

Zhang, T., & Zhang, K. (2022). Eventual strong detectability of labeled weighted automata over monoids. In *16th IFAC workshop on discrete event systems, vol. 55* (pp. 270–275).

Zhang, K., Zhang, L., & Xie, L. (2020). Discrete-time and discrete-space dynamical systems. In *Communications and control engineering*. Springer International Publishing.

Zhao, J., Yin, X., & Li, S. (2022). A unified framework for verification of observational properties for partially-observed discrete-event systems. https://arxiv.org/pdf/2205.01392.pdf.